

PROGRAMACIÓN DE DISPOSITIVOS MÓVILES · ANDROID STUDIO · JAVA

Componentes Avanzados en el Componente Móvil

Guía de Actividades · Punto 3.8

Documento de estudio, desarrollo y sustentación

Continuación del proyecto **MiAgenda** (guía 3.7)

RecyclerView · Adapter-ViewHolder · Intents · Fragments
código completo · guion de exposición · banco de preguntas

Cómo usar este documento

El punto 3.8 tiene una particularidad evaluativa que el 3.7 no tenía: **15 % de la nota es sustentación individual dentro de la pareja, mediante preguntas cruzadas del docente**. Es decir, se evalúa explícitamente si cada integrante entendió lo que la pareja entregó.

Además, la propia guía advierte cuál es el contenido crítico:

“Cada pareja debe explicar con precisión la diferencia entre un Intent explícito y uno implícito, y qué ocurre con el estado de un Fragment cuando se rota el dispositivo. **Este es el contenido que con mayor frecuencia se reproduce sin comprensión.**”

Ese aviso es literalmente el enunciado de las dos preguntas que van a caer. Están respondidas a fondo en los apartados 2.6 y 2.7 y repasadas en el banco de preguntas.

Pasada	Qué hacer	Tiempo	Resultado
1ª — Comprensión	Capítulo 2 completo, sin memorizar. Dibujar a mano los dos ciclos de vida.	75 min	Entender por qué el reciclado es necesario y por qué rotar destruye la Activity.
2ª — Construcción	Teclear los capítulos 4 y 5 en Android Studio, en el orden dado.	5-6 h	Aplicación funcionando y errores reales enfrentados.
3ª — Retención	Capítulos 8 (guion) y 9 (banco de preguntas), tapando las respuestas.	90 min	Poder sustentar sin leer.

LAS DOS IDEAS QUE SOSTIENEN TODO EL PUNTO 3.8

1. Reciclar en lugar de crear. Una lista de mil elementos no crea mil vistas: crea aproximadamente las que caben en pantalla más un pequeño margen, y las reutiliza cambiándoles el contenido. El patrón Adapter-ViewHolder es el mecanismo que hace eso posible, y `RecyclerView` lo vuelve obligatorio por diseño de su API.

2. Los componentes no se llaman: se solicitan. No se crea una Activity con `new`. Se declara una intención (`Intent`) y es el sistema quien decide qué componente la atiende, lo instancia y gestiona su ciclo de vida. Ese nivel de indirección es lo que permite que una aplicación comparta una tarea con WhatsApp sin conocer WhatsApp.

Advertencia previa: el nombre del proyecto

La guía del punto 3.7 indica crear el proyecto `MiAgendaINTJEM`; esta guía del 3.8 afirma ser continuación de `MiAgendaUPDS`. **Es el mismo proyecto con dos nombres distintos en los enunciados.**

QUÉ HACER

Consultarlo con el docente antes de continuar. Renombrar el paquete a mitad de camino obliga a tocar `applicationId`, `namespace`, la estructura de carpetas bajo `java/` y todos los `import`, y contamina el historial de Git justo donde se evalúa la trazabilidad de autoría.

Este documento mantiene `com.example.miagendaintjem` por continuidad con la guía 3.7. Si se confirma “UPDS”, se cambia el nombre del proyecto, el `applicationId` y `@string/app_name`; el código de este documento no cambia salvo la línea `package`.

Aviso sobre el bloque teórico

En el archivo PDF entregado por la cátedra, el apartado **2.2 “Bloque teórico” del punto 3.8 aparece con el encabezado pero sin contenido**: no hay actividades AT-3.8.1 ni AT-3.8.2 enunciadas. Sin embargo:

- La ponderación asigna **25 %** a “Actividades teóricas (2)”.
- La estructura del informe pide “cuadro comparativo, ciclo de vida y síntesis de la defensa técnica”.

A partir de esas dos pistas, el capítulo 3 de este documento **reconstruye** las dos actividades teóricas más probables. **Hay que confirmarlas con el docente**, porque una reconstrucción razonable no es el enunciado oficial.

Fundamentos conceptuales

De widgets elementales a dos familias de componentes

Familia	Qué son	Ejemplos de esta guía
Componentes avanzados de interfaz	Vistas y contenedores complejos que viven dentro de una Activity	RecyclerView, CardView, Fragment, BottomNavigationView, FloatingActionButton, Toolbar
Componentes de aplicación	Puntos de entrada que el sistema operativo instancia y gobierna	Activity, Service, BroadcastReceiver, ContentProvider

La distinción importa: un `RecyclerView` lo crea el programador inflando un layout. Una `Activity` **no la crea el programador**: la crea el sistema, en respuesta a un `Intent`, y controla su ciclo de vida. Confundir ambas familias es el origen de la mayoría de los errores de ciclo de vida.

Por qué existe el reciclado de vistas

El costo real de una vista

Crear una vista en Android implica tres operaciones caras:

- Inflar el XML** — leer el layout binario, instanciar cada clase por reflexión, aplicar atributos. Es, con diferencia, lo más costoso.
- `findViewById` — recorrer el árbol de vistas buscando un identificador. Es un recorrido, no un acceso indexado.
- Medir y disponer** — calcular tamaño y posición de cada vista de la jerarquía.

El cálculo que demuestra el argumento

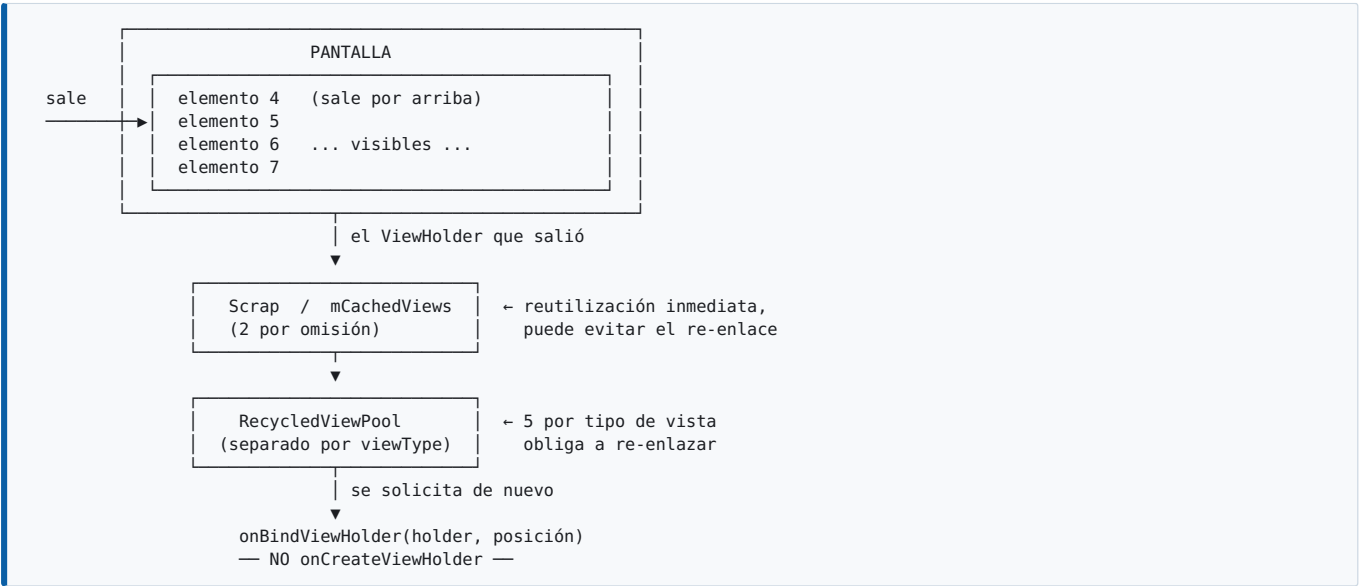
Supóngase una lista de **1 000 tareas**, con **4 vistas** por elemento (título, materia, fecha, indicador), de las cuales **10 caben en pantalla**.

Enfoque	Vistas infladas	Llamadas a <code>findViewById</code>	Memoria
<code>LinearLayout</code> dentro de <code>ScrollView</code>	1 000 elementos = 4 000 vistas	4 000	Todas vivas simultáneamente
<code>RecyclerView</code> con <code>ViewHolder</code>	≈ 13 elementos = 52 vistas	52	Solo las visibles + margen

LA CIFRA QUE HAY QUE PODER DECIR EN LA SUSTENTACIÓN

“Con mil elementos y diez visibles, `RecyclerView` crea alrededor de **trece** `ViewHolder`, no mil. `findViewById` se ejecuta **52 veces en lugar de 4 000**, porque se ejecuta una vez por vista *creada* y no una vez por vista *mostrada*. Al desplazar la lista no se crea nada: se cambia el contenido de los trece que ya existen.”

Cómo recicla internamente



Consecuencia para el programador: `onCreateViewHolder` se llama pocas veces; `onBindViewHolder` se llama constantemente durante el desplazamiento. Por eso `onBindViewHolder` debe ser **barato**: solo asignar datos. Todo lo caro —inflar, buscar vistas, registrar escuchadores— va en `onCreateViewHolder` y en el constructor del ViewHolder.

El patrón Adapter-ViewHolder

Pieza	Responsabilidad	Analogía
Modelo (Tarea)	Contener los datos	El contenido
Adapter	Traducir entre la lista de datos y las vistas; decidir cuántos elementos hay y cómo se enlaza cada uno	El traductor
ViewHolder	Guardar las referencias a las vistas de un elemento, para no volver a buscarlas	La ficha reutilizable
LayoutManager	Decidir la disposición geométrica (lineal, cuadrícula, escalonada) y cuándo reciclar	El maquetador
ItemAnimator	Animar inserciones, eliminaciones y cambios	El animador
ItemDecoration	Dibujar separadores, márgenes y adornos	El decorador

DIFERENCIA HISTÓRICA CON LISTVIEW (PREGUNTA MUY PROBABLE)

En `ListView`, el reciclado existía mediante el parámetro `convertView` de `getView()`, pero era **opcional**: si el programador lo ignoraba e inflaba siempre una vista nueva, la lista funcionaba —lenta, pero funcionaba—. El patrón ViewHolder era una *convención*, no una obligación.

`RecyclerView` hace que la API **imponga** el patrón: `onCreateViewHolder` y `onBindViewHolder` son métodos *separados* y es el framework quien decide cuándo llamar a cada uno. El programador ya no puede saltarse el reciclado aunque quiera.

Además, `RecyclerView` separó responsabilidades que en `ListView` estaban fundidas: la disposición pasó al `LayoutManager` (por eso cambiar de lista a cuadrícula es una línea), las animaciones al `ItemAnimator` y los separadores al `ItemDecoration`.

Los tres métodos obligatorios

Método	Cuándo lo llama el framework	Qué debe hacer	Qué NO debe hacer
<code>onCreateViewHolder</code>	Pocas veces: cuando no hay ViewHolder reutilizable	Inflar <code>item_*.xml</code> , crear el ViewHolder, registrar escuchadores	Usar la posición (todavía no existe)
<code>onBindViewHolder</code>	Constantemente, en cada desplazamiento	Asignar los datos de <code>position</code> a las vistas ya existentes	Inflar vistas , llamar a <code>findViewById</code> , registrar escuchadores nuevos
<code>getItemCount</code>	Cuando el layout manager necesita saber el total	Devolver <code>lista.size()</code>	Devolver un valor fijo o recalculado costosamente

Notificar cambios: el método correcto

Método	Efecto	Cuándo usarlo
<code>notifyDataSetChanged()</code>	Invalida todo ; sin animaciones; se pierde la posición de desplazamiento	Solo cuando cambió la lista entera
<code>notifyItemInserted(pos)</code>	Inserta con animación	Al añadir un elemento
<code>notifyItemRemoved(pos)</code>	Elimina con animación	Al borrar un elemento
<code>notifyItemChanged(pos)</code>	Re-enlaza solo ese elemento	Al editar un elemento
<code>DiffUtil</code>	Calcula el diferencial mínimo entre dos listas y emite las notificaciones exactas	Listas que se reemplazan por completo

ERROR MUY COMÚN Y FÁCIL DE DETECTAR

Usar `notifyDataSetChanged()` para todo “porque funciona”. Funciona, pero desactiva las animaciones del `ItemAnimator`, obliga a re-enlazar todos los elementos visibles y, en listas largas con imágenes, produce parpadeo visible.

En la sustentación conviene poder decir: “Usamos `notifyItemInserted` y `notifyItemRemoved` porque conservan las animaciones y solo re-enlazan lo que cambió. `notifyDataSetChanged` lo reservamos para un reemplazo completo de la lista.”

Los cuatro componentes de aplicación

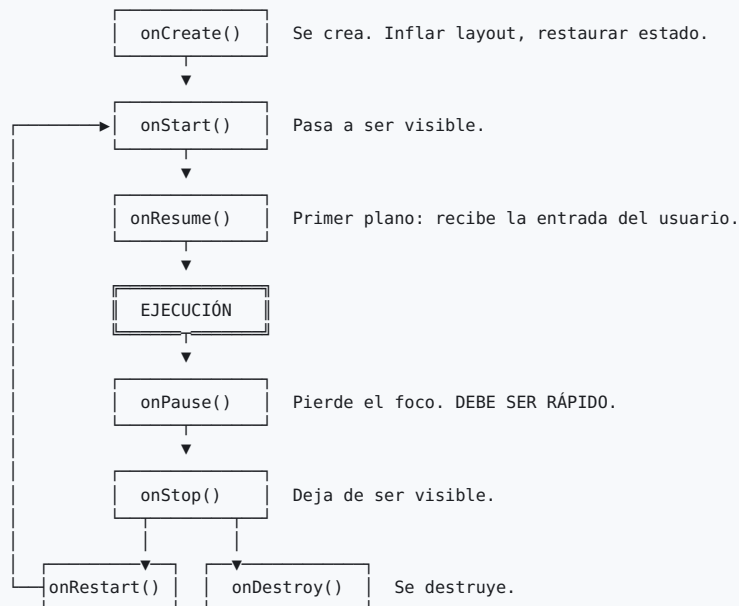
Componente	Para qué sirve	Etiqueta en el Manifest	Cómo se activa
Activity	Una pantalla con interfaz. Punto de entrada del usuario.	<code><activity></code>	<code>startActivity(Intent)</code>
Service	Trabajo prolongado sin interfaz (descarga, reproducción de audio).	<code><service></code>	<code>startService()</code> / <code>bindService()</code>
BroadcastReceiver	Responder a anuncios del sistema o de otras apps (batería baja, arranque, conectividad).	<code><receiver></code>	Envío de un <code>Intent</code> de difusión
ContentProvider	Exponer datos estructurados a otras aplicaciones (contactos, galería).	<code><provider></code> con <code>authority</code>	<code>ContentResolver</code> mediante una <code>Uri</code>

Detalles que el docente puede preguntar

- Solo `ContentProvider` se activa con `ContentResolver`; los otros tres se activan con un `Intent`.
- Desde **Android 12 (API 31)**, todo componente con `<intent-filter>` debe declarar explícitamente `android:exported="true"` o `"false"`. Omitirlo impide instalar la aplicación.
- Desde **Android 8 (API 26)**, la mayoría de las difusiones implícitas ya **no** pueden registrarse en el Manifest: hay que registrar el receptor desde código con `registerReceiver()`. La razón fue el consumo de batería: cada difusión despertaba decenas de aplicaciones dormidas.
- Desde **Android 14 (API 34)**, un servicio en primer plano debe declarar su `android:foregroundServiceType`.

En esta guía se usa **un solo** componente de aplicación: `Activity` (la principal y la de detalle). Los `Fragment` **no** son componentes de aplicación: son piezas reutilizables de interfaz que viven **dentro** de una `Activity`. Confundirlos es un error frecuente y fácil de detectar.

Ciclo de vida de una Activity



Método	Cuándo	Qué hacer aquí
<code>onCreate(Bundle)</code>	Una vez, al crearse	<code>setContentView</code> , enlazar vistas, restaurar estado del <code>Bundle</code>
<code>onStart()</code>	Cada vez que pasa a visible	Registrar observadores de datos
<code>onResume()</code>	Cada vez que recibe el foco	Reanudar animaciones, abrir cámara
<code>onPause()</code>	Al perder el foco	Pausar animaciones. Debe ser rápido : la siguiente pantalla no aparece hasta que termine
<code>onStop()</code>	Al dejar de ser visible	Guardar datos costosos, liberar recursos
<code>onSaveInstanceState(Bundle)</code>	Antes de una posible destrucción	Guardar el estado transitorio de la interfaz
<code>onDestroy()</code>	Al destruirse	Última limpieza

POR QUÉ ROTAR DESTRUYE LA ACTIVITY (Y NO ES UN ERROR DE DISEÑO)

Al girar el dispositivo cambia el objeto `Configuration`: orientación, ancho y alto disponibles y, potencialmente, el calificador aplicable. Como **los recursos se eligen en función de la `Configuration`**, la Activity debe volver a cargarlos: `layout-land` en vez de `layout`, `values-sw600dp` en vez de `values`.

La manera más simple y confiable de garantizar que todo se recargue coherentemente es **destruir y recrear** la Activity. Es la misma lógica del punto 3.7 vista desde el otro lado: el sistema puede adaptar la aplicación sin código del programador precisamente porque se permite rehacerla entera.

Secuencia exacta al rotar: `onPause` → `onStop` → `onSaveInstanceState` → `onDestroy` → `onCreate` → `onStart` → `onRestoreInstanceState` → `onResume`.

Ciclo de vida de un Fragment

Un Fragment tiene **dos ciclos superpuestos**: el del propio fragmento y el de su vista. Entender esa separación es la clave del punto.

```

onAttach()      Se asocia a la Activity. Ya hay Context.
  ▼
onCreate()      Se crea el fragmento. NO hay vista todavía.
  ▼
onCreateView()  Se INFLA la vista. Solo inflar y devolver.
  ▼
onViewCreated() La vista ya existe. Aquí se enlazan vistas y escuchadores.
  ▼
onStart() → onResume() == ACTIVO == onPause() → onStop()
  ▼
onDestroyView() Se DESTRUYE la vista. El fragmento SIGUE VIVO.
  ▼
onDestroy()     Se destruye el fragmento.
  ▼
onDetach()      Se desvincula de la Activity.

```

LA ASIMETRÍA QUE HAY QUE SABER EXPLICAR

Un Fragment puede pasar por `onCreateView` y `onDestroyView` **varias veces** sin que se ejecute `onDestroy`. Ocurre, por ejemplo, cuando se añade a la pila de retroceso: su vista se destruye para liberar memoria, pero el objeto Fragment permanece.

Consecuencia práctica: toda referencia a vistas debe anularse en `onDestroyView()`. Conservar un `TextView` en un campo después de `onDestroyView` es una **fuga de memoria** clásica: se retiene el árbol de vistas entero y, con él, el Context de la Activity.

Activity frente a Fragment

Criterio	Activity	Fragment
¿Componente de aplicación?	Sí	No: es parte de la interfaz
Declaración en el Manifest	Obligatoria	No se declara
Quién la crea	El sistema , vía <code>Intent</code>	El programador , vía <code>FragmentManager</code>
Quién gobierna su ciclo de vida	El sistema	La Activity anfitriona, a través del <code>FragmentManager</code>
Reutilización	Baja: es una pantalla completa	Alta: la misma pieza sirve en teléfono y tablet
Ciclo de vida	Uno	Dos: el del fragmento y el de su vista
Comunicación	<code>Intent</code> + extras	Argumentos (<code>Bundle</code>), <code>ViewModel</code> compartido, <code>FragmentResult</code>
Coste de la transición	Alto (nueva ventana)	Bajo (cambio dentro de la misma ventana)

Regla práctica: cuando la navegación es entre secciones de un mismo nivel —como las tres pestañas del `BottomNavigationView`— se usan Fragments. Cuando es un salto a otro contexto —abrir el detalle de una tarea— se usa una Activity nueva, como pide la consigna.

Intent explícito frente a implícito

LA GUÍA LO DICE LITERALMENTE

“Cada pareja debe explicar con precisión la diferencia entre un Intent explícito y uno implícito... Este es el contenido que con mayor frecuencia se reproduce sin comprensión.”

Esta pregunta va a caer. La respuesta que sigue es la completa.

	Intent explícito	Intent implícito
Qué declara	Quién debe atenderlo: la clase concreta	Qué se quiere hacer: una acción y un tipo de dato
Ejemplo	<code>new Intent(this, DetalleTareaActivity.class)</code>	<code>new Intent(Intent.ACTION_SEND).setType("text/plain")</code>
Cómo lo resuelve el sistema	Directamente: el componente está nombrado	Resolución de intención: compara la acción, los datos y las categorías contra los <code><intent-filter></code> de todas las apps instaladas
Resultado posible	Un componente exacto	Cero, uno o varios candidatos; con varios, aparece el selector
Requiere <code><intent-filter></code>	No	Sí , en el componente que desee atenderlo
Uso típico	Navegación dentro de la propia aplicación	Delegar en otras aplicaciones (compartir, llamar, abrir un mapa)
Riesgo	Ninguno	Que no haya ninguna app capaz de atenderlo

La regla de seguridad que conviene mencionar

Para componentes internos hay que usar siempre Intents explícitos. Un Intent implícito es resuelto por el sistema contra *todas* las aplicaciones instaladas; si otra aplicación declara un `<intent-filter>` coincidente, podría interceptar los datos. Por eso los Services, desde Android 5.0, no pueden arrancarse con Intents implícitos.

Detalle adicional para el Intent implícito

Desde **Android 11 (API 30)** existe la *visibilidad de paquetes*: una aplicación no puede enumerar libremente las demás. Si se necesita comprobar previamente si hay alguien capaz de atender el Intent con `intent.resolveActivity(getPackageManager())`, hay que declarar un bloque `<queries>` en el Manifest. Usar `Intent.createChooser()` evita el problema, porque el selector lo presenta el propio sistema.

Estado de un Fragment al rotar

SEGUNDA PREGUNTA ANUNCIADA POR LA GUÍA

“...y qué ocurre con el estado de un Fragment cuando se rota el dispositivo.”

Qué ocurre, paso a paso:

1. Al rotar, el sistema **destruye la Activity** anfitriona.
2. Antes de destruirla, el `FragmentManager` **guarda el estado de sus fragmentos** dentro del `Bundle` de `onSaveInstanceState` de la Activity: qué fragmentos había, en qué contenedor y con qué argumentos.
3. La Activity se recrea. En `super.onCreate(savedInstanceState)`, el `FragmentManager` **restaura automáticamente** esos fragmentos: crea instancias nuevas de las mismas clases y les pasa su `Bundle` de estado.
4. **Los campos de la instancia anterior se pierden.** El objeto Fragment restaurado es un objeto **nuevo**: se llama al constructor vacío y luego a `onCreate(savedInstanceState)`. Una lista guardada en un campo aparece vacía.
5. La vista se destruye (`onDestroyView`) y se vuelve a inflar (`onCreateView`). El estado de las vistas **con `android:id`** se restaura solo (texto de un `EditText`, posición de desplazamiento, pestaña seleccionada).

El error número uno

```
// INCORRECTO: al rotar se añade un SEGUNDO fragmento encima del restaurado
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState); // el FragmentManager ya restauró
    setContentView(R.layout.activity_main);
    getSupportFragmentManager().beginTransaction()
        .replace(R.id.contenedorFragmentos, new TareasFragment())
        .commit(); // y aquí se añade otro
}
```

```
// CORRECTO
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    if (savedInstanceState == null) { // solo en la PRIMERA creación
        getSupportFragmentManager().beginTransaction()
            .replace(R.id.contenedorFragmentos, new TareasFragment())
            .commit();
    }
}
```

Síntoma si falta la guarda: al rotar, el contenido se ve superpuesto o duplicado, los clics responden al fragmento equivocado y, con listas, los elementos aparecen dos veces.

Cómo conservar los datos propios

Mecanismo	Sobrevive a rotación	Sobrevive a muerte del proceso	Comentario
Campo de la instancia	No	No	Se pierde: la instancia es nueva
<code>onSaveInstanceState(Bundle)</code>	Sí	Sí	Limitado a datos pequeños y serializables o <code>Parcelable</code>
<code>ViewModel</code>	Sí	No	Pensado exactamente para esto; requiere la dependencia de lifecycle
Base de datos / preferencias	Sí	Sí	Para datos que deben persistir de verdad

En esta guía se usa `onSaveInstanceState` con `putParcelableArrayList`, porque `Tarea` ya implementa `Parcelable` para poder viajar en el Intent, y no exige dependencias nuevas.

Bloque teórico (reconstruido)

Recordatorio: el apartado 2.2 del PDF de la cátedra llegó **sin contenido**. Lo que sigue se deduce de la ponderación (25 % para dos actividades teóricas) y de la estructura del informe, que pide “cuadro comparativo, ciclo de vida y síntesis de la defensa técnica”. **Confirmar con el docente.**

AT-3.8.1 (propuesta) · Cuadros comparativos

Cuadro 1 · ListView frente a RecyclerView

Criterio	ListView	RecyclerView
Patrón ViewHolder	Opcional, por convención	Obligatorio por diseño de la API
Reciclado	Mediante <code>convertView</code> , ignorable	Automático y no evitable
Disposición	Solo vertical	Delegada al <code>LayoutManager</code> : lineal, cuadrícula, escalonada
Animaciones de elemento	No las tiene	<code>ItemAnimator</code> integrado
Separadores	<code>android:divider</code>	<code>ItemDecoration</code> (más flexible)
Actualizaciones parciales	<code>notifyDataSetChanged()</code> únicamente	<code>notifyItemInserted/Removed/Changed</code> , <code>DiffUtil</code>
Eventos de clic	<code>setOnClickListener</code> en la lista	Se implementan en el ViewHolder o mediante interfaz propia
Paquete	<code>android.widget</code> (framework)	<code>androidx.recyclerview</code> (biblioteca)
Complejidad inicial	Menor	Mayor, pero escala mucho mejor

Conclusión para el informe: `RecyclerView` no es “un `ListView` mejorado”: es un rediseño que **separa responsabilidades** (datos, vista, disposición, animación, decoración) y convierte en obligatorio lo que antes era una buena práctica opcional.

Cuadro 2 · Activity frente a Fragment

Véase la tabla del apartado 2.5. Los tres puntos que no pueden faltar:

- 1. La Activity **es** un componente de aplicación y se declara en el Manifest; el Fragment **no**.
- 2. La Activity la instancia el **sistema** a partir de un Intent; el Fragment lo instancia el **programador** a través del `FragmentManager` .
- 3. El Fragment tiene **dos** ciclos de vida superpuestos: el suyo y el de su vista.

Cuadro 3 · Los cuatro componentes de aplicación

Componente	Interfaz	Etiqueta Manifest	Activación	Ejemplo real
Activity	Sí	<code><activity></code>	<code>startActivity(Intent)</code>	Pantalla de detalle de tarea
Service	No	<code><service></code>	<code>startService</code> / <code>bindService</code>	Sincronizar tareas en segundo plano
BroadcastReceiver	No	<code><receiver></code>	Difusión de un Intent	Reprogramar recordatorios tras reiniciar
ContentProvider	No	<code><provider></code>	<code>ContentResolver</code> + <code>Uri</code>	Exponer las tareas a un widget del escritorio

AT-3.8.2 (propuesta) · Ciclo de vida y defensa técnica del reciclado

Parte A · Diagramas de ciclo de vida

Reproducir a mano (no copiar los de la documentación) los dos diagramas del capítulo 2 y añadir una **tabla de correspondencia** de qué se hace en cada método, aplicada al proyecto concreto:

Método	En MainActivity	En TareasFragment
onCreate	setContentView, Toolbar, navegación, guarda savedInstanceState == null	Registrar el <code>ActivityResultLauncher</code>
onCreateView	—	Inflar <code>fragment_tareas.xml</code>
onViewCreated	—	Configurar <code>RecyclerView</code> , adaptador y FAB
onSaveInstanceState	—	Guardar la lista de tareas como <code>ParcelableArrayList</code>
onDestroyView	—	Anular referencias a vistas para evitar fugas

Parte B · Síntesis de la defensa técnica

Texto sugerido para el informe:

El reciclado de vistas responde a una asimetría: el número de elementos de una lista es arbitrario, pero el número de elementos **visibles** está acotado por el tamaño de la pantalla. Crear una vista por elemento hace que el costo crezca linealmente con los datos, cuando en realidad la pantalla no puede mostrar más de una docena a la vez.

`RecyclerView` explota esa acotación: mantiene un conjunto de `ViewHolder` proporcional a lo visible —no a los datos— y, cuando un elemento sale de pantalla, devuelve su `ViewHolder` a un depósito (`RecycledViewPool`) para reutilizarlo en el elemento que entra. El patrón `ViewHolder` complementa esa estrategia almacenando las referencias a las vistas hijas, de modo que `findViewById` —que es un recorrido del árbol de vistas, no un acceso indexado— se ejecute una sola vez por vista creada y no una vez por elemento mostrado.

En una lista de mil tareas con diez visibles, esto significa inflar alrededor de trece elementos en lugar de mil, y ejecutar `findViewById` unas cincuenta veces en lugar de cuatro mil. La consecuencia de diseño es que `onCreateViewHolder` debe concentrar todo el trabajo costoso —inflado, búsqueda de vistas y registro de escuchadores— mientras que `onBindViewHolder`, que se ejecuta en cada desplazamiento, debe limitarse a asignar datos.

AP-3.8.1 · Lista de tareas con RecyclerView y CardView

Modalidad: práctica · 120 min · en pareja

Restricción: reutilizar exclusivamente los recursos definidos en la guía del punto 3.7.

CRITERIOS DE RECHAZO (LA PRÁCTICA SE DEVUELVE SIN CALIFICAR)

1. Si el adaptador infla una vista nueva dentro de `onBindViewHolder`.
2. Si se utilizan valores literales en `item_tarea.xml`.
3. Si la lista se construye con un `LinearLayout` dentro de un `ScrollView` en lugar de `RecyclerView`.

Los tres están explicados y justificados en el capítulo 6. Conviene leerlo *antes* de escribir el código, no después.

Paso 0 · Dependencias y recursos nuevos

```
dependencies {
    implementation 'androidx.appcompat:appcompat:1.7.0'
    implementation 'com.google.android.material:material:1.12.0'
    implementation 'androidx.constraintlayout:constraintlayout:2.1.4'
    implementation 'androidx.recyclerview:recyclerview:1.3.2'
    implementation 'androidx.core:core:1.13.1'
}
```

`MaterialCardView` y `FloatingActionButton` vienen dentro de la biblioteca `material`; no hace falta añadir `cardview` por separado.

Recursos añadidos a los de la guía 3.7

```
<!-- res/values/colors.xml : se AÑADEN a los existentes -->
<color name="prioridad_alta">#BA1A1A</color>
<color name="prioridad_media">#E08600</color>
<color name="prioridad_baja">#1D7A3C</color>
```

```
<!-- res/values/dimens.xml : se AÑADEN a los existentes -->
<dimen name="elevacion_tarjeta">2dp</dimen>
<dimen name="ancho_indicador">6dp</dimen>
<dimen name="texto_item_titulo">18sp</dimen>
<dimen name="texto_item_secundario">14sp</dimen>
<dimen name="separacion_indicador">12dp</dimen>
```

```
<!-- res/values/strings.xml : se AÑADEN a los existentes -->
<string name="prioridad_alta">Alta</string>
<string name="prioridad_media">Media</string>
<string name="prioridad_baja">Baja</string>

<string name="formato_fecha_entrega">Entrega: %1$s</string>
<string name="desc_indicador_prioridad">Indicador de prioridad</string>

<string name="fab_nueva_tarea">Nueva tarea</string>
<string name="dialogo_nueva_titulo">Nueva tarea</string>
<string name="campo_titulo">Título de la tarea</string>
<string name="accion_guardar">Guardar</string>
<string name="accion_cancelar">Cancelar</string>
<string name="accion_eliminar">Eliminar</string>

<string name="dialogo_eliminar_titulo">Eliminar tarea</string>
<string name="dialogo_eliminar_mensaje">¿Deseas eliminar «%1$s»? Esta acción no se puede deshacer.</string>
<string name="msg_tarea Eliminada">Tarea eliminada</string>
<string name="error_titulo_vacio">El título no puede estar vacío</string>
```

```

<!-- res/values/arrays.xml : NUEVO archivo -->
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string-array name="materias">
        <item>Programación de Dispositivos Móviles</item>
        <item>Base de Datos II</item>
        <item>Ingeniería de Software</item>
        <item>Redes de Computadoras</item>
        <item>Inteligencia Artificial</item>
    </string-array>

    <string-array name="tareas_demo">
        <item>Guía 3.7: sistema de recursos</item>
        <item>Normalizar el modelo relacional</item>
        <item>Diagrama de casos de uso</item>
        <item>Práctica de subredes</item>
        <item>Informe de búsqueda A*</item>
        <item>Guía 3.8: RecyclerView</item>
        <item>Respaldo y recuperación</item>
        <item>Documento de arquitectura</item>
    </string-array>

    <string-array name="prioridades">
        <item>@string/prioridad_alta</item>
        <item>@string/prioridad_media</item>
        <item>@string/prioridad_baja</item>
    </string-array>
</resources>

```

DETALLE QUE DEMUESTRA DOMINIO DEL PUNTO 3.7

El string-array prioridades no repite los textos: **referencia** a las cadenas ya definidas con @string/... . Así, al traducir a inglés en values-en basta con traducir las tres cadenas; el array se resuelve solo.

Es exactamente la continuidad entre las dos guías que la cátedra espera ver: los recursos del 3.7 no se abandonan, se reutilizan.

Paso 1 • modelo/Prioridad.java

```

package com.example.miagendaintjem.modelo;

import androidx.annotation.ColorRes;
import androidx.annotation.StringRes;

import com.example.miagendaintjem.R;

/**
 * Prioridad de una tarea. Cada valor conoce el RECURSO que lo representa,
 * nunca el color ni el texto concretos: se mantiene la disciplina del punto 3.7.
 */
public enum Prioridad {

    ALTA (R.color.prioridad_alta, R.string.prioridad_alta),
    MEDIA (R.color.prioridad_media, R.string.prioridad_media),
    BAJA (R.color.prioridad_baja, R.string.prioridad_baja);

    @ColorRes private final int colorRes;
    @StringRes private final int etiquetaRes;

    Prioridad(@ColorRes int colorRes, @StringRes int etiquetaRes) {
        this.colorRes = colorRes;
        this.etiquetaRes = etiquetaRes;
    }

    @ColorRes public int getColorRes() { return colorRes; }
    @StringRes public int getEtiquetaRes() { return etiquetaRes; }
}

```

Las anotaciones @ColorRes y @StringRes no cambian la ejecución: son **contratos verificados por Lint**. Si alguien pasara R.string.algo donde se espera @ColorRes, Android Studio lo marca como error antes de compilar. Mencionarlo en la sustentación es un punto a favor barato y real.

Paso 2 • modelo/Tarea.java

```

package com.example.miagendaintjem.modelo;

import android.os.Parcel;
import android.os.Parcelable;

import androidx.annotation.NonNull;

public class Tarea implements Parcelable {

    private final int id;
    private String titulo;
    private String materia;
    private String fechaEntrega;
    private Prioridad prioridad;

    public Tarea(int id, String titulo, String materia,
                 String fechaEntrega, Prioridad prioridad) {
        this.id = id;
        this.titulo = titulo;
        this.materia = materia;
        this.fechaEntrega = fechaEntrega;
        this.prioridad = prioridad;
    }

    // ----- Parcelable -----
    // REGLA CRÍTICA: el orden de lectura debe ser IDÉNTICO al de escritura.

    protected Tarea(Parcel origen) {
        id = origen.readInt();
        titulo = origen.readString();
        materia = origen.readString();
        fechaEntrega = origen.readString();
        prioridad = Prioridad.valueOf(origen.readString());
    }

    @Override
    public void writeToParcel(@NonNull Parcel destino, int flags) {
        destino.writeInt(id);
        destino.writeString(titulo);
        destino.writeString(materia);
        destino.writeString(fechaEntrega);
        destino.writeString(prioridad.name());
    }

    @Override
    public int describeContents() { return 0; }

    public static final Creator<Tarea> CREATOR = new Creator<Tarea>() {
        @Override public Tarea createFromParcel(Parcel origen) { return new Tarea(origen); }
        @Override public Tarea[] newArray(int tamano) { return new Tarea[tamano]; }
    };

    // ----- Accesorios -----
    public int getId() { return id; }
    public String getTitulo() { return titulo; }
    public String getMateria() { return materia; }
    public String getFechaEntrega() { return fechaEntrega; }
    public Prioridad getPrioridad() { return prioridad; }

    public void setTitulo(String titulo) { this.titulo = titulo; }
    public void setMateria(String materia) { this.materia = materia; }
    public void setFechaEntrega(String fecha) { this.fechaEntrega = fecha; }
    public void setPrioridad(Prioridad prioridad) { this.prioridad = prioridad; }
}

```

POR QUÉ PARCELABLE Y NO SERIALIZABLE

Ambos permiten que un objeto viaje dentro de un `Intent`, pero funcionan de manera distinta:

Serializable es un marcador de Java puro. La serialización se resuelve por **reflexión** en tiempo de ejecución, lo que genera muchos objetos temporales y presiona al recolector de basura. Es una línea de código y mucho trabajo oculto.

Parcelable es la solución de Android. El programador escribe explícitamente qué campos se escriben y en qué orden. Sin reflexión, sin objetos temporales: en Android es del orden de diez veces más rápido.

La trampa: el orden de `writeToParcel` y el del constructor `Tarea(Parcel)` deben coincidir **exactamente**. Si se invierten dos campos, no hay error de compilación: la aplicación lee basura o lanza una excepción de tipo en ejecución. Es el error más difícil de encontrar de todo el punto 3.8.

```

<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/tarjetaTarea"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginStart="@dimen/espaciado_medio"
    android:layout_marginEnd="@dimen/espaciado_medio"
    android:layout_marginTop="@dimen/espaciado_pequeno"
    android:clickable="true"
    android:focusable="true"
    android:foreground="?attr/selectableItemBackground"
    app:cardCornerRadius="@dimen/radio_medio"
    app:cardElevation="@dimen/elevacion_tarjeta">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="@dimen/espaciado_medio">

        <!-- Franja de color según la prioridad -->
        <View
            android:id="@+id/indicadorPrioridad"
            android:layout_width="@dimen/ancho_indicador"
            android:layout_height="0dp"
            android:contentDescription="@string/desc_indicador_prioridad"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent" />

        <TextView
            android:id="@+id/tvTitulo"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginStart="@dimen/separacion_indicador"
            android:ellipsize="end"
            android:maxLines="2"
            android:textColor="?attr/colorOnSurface"
            android:textSize="@dimen/texto_item_titulo"
            android:textStyle="bold"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toEndOf="@id/indicadorPrioridad"
            app:layout_constraintTop_toTopOf="parent" />

        <TextView
            android:id="@+id/tvMateria"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginStart="@dimen/separacion_indicador"
            android:layout_marginTop="@dimen/espaciado_pequeno"
            android:ellipsize="end"
            android:maxLines="1"
            android:textColor="?attr/colorPrimary"
            android:textSize="@dimen/texto_item_secundario"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toEndOf="@id/indicadorPrioridad"
            app:layout_constraintTop_toBottomOf="@id/tvTitulo" />

        <TextView
            android:id="@+id/tvFecha"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginStart="@dimen/separacion_indicador"
            android:layout_marginTop="@dimen/espaciado_pequeno"
            android:textSize="@dimen/texto_item_secundario"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintStart_toEndOf="@id/indicadorPrioridad"
            app:layout_constraintTop_toBottomOf="@id/tvMateria" />

    </androidx.constraintlayout.widget.ConstraintLayout>
</com.google.android.material.card.MaterialCardView>

```

Qué hay que saber defender de este archivo

Elemento	Justificación
Ningún literal	Todos los tamaños, márgenes y radios son <code>@dimen</code> ; los colores son <code>?attr</code> o se asignan por código desde <code>@color</code> . Cumple el criterio de rechazo 2.
<code>?attr/colorOnSurface</code>	Se declara el rol del color, no el color. En modo oscuro cambia solo, gracias al tema Material 3 de la guía 3.7.
<code>android:layout_height="0dp"</code> en el indicador	Con restricciones arriba y abajo, <code>0dp</code> hace que la franja ocupe toda la altura de la tarjeta, sea cual sea el largo del título.
<code>foreground="?attr/selectableItemBackground"</code>	Produce el efecto visual de pulsación (<i>ripple</i>) sin una línea de Java.
<code>ellipsize + maxLines</code>	Un título largo no rompe la tarjeta: se recorta con puntos suspensivos. Las tarjetas mantienen altura homogénea, lo que además favorece el reciclado.
Sin texto en los <code>TextView</code>	El texto lo pone <code>onBindViewHolder</code> . En el editor puede usarse <code>tools:text="Ejemplo"</code> para previsualizar, porque <code>tools:</code> no llega al APK.

Paso 4 • adaptador/TareaAdapter.java

```
package com.example.miagendaintjem.adaptador;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.core.content.ContextCompat;
import androidx.recyclerview.widget.RecyclerView;

import com.example.miagendaintjem.R;
import com.example.miagendaintjem.modelo.Tarea;

import java.util.List;

public class TareaAdapter
    extends RecyclerView.Adapter<TareaAdapter.TareaViewHolder> {

    /** Interfaz de escucha propia: el adaptador NO decide qué ocurre al pulsar. */
    public interface OnTareaClickListener {
        void onTareaClick(Tarea tarea, int posicion);
        void onTareaLongClick(Tarea tarea, int posicion);
    }

    private final List<Tarea> lista;
    private final OnTareaClickListener listener;

    public TareaAdapter(List<Tarea> lista, OnTareaClickListener listener) {
        this.lista = lista;
        this.listener = listener;
    }

    // ----- 1. CREAR: se llama POCAS veces -----
    @NonNull
    @Override
    public TareaViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
        View vista = LayoutInflater.from(parent.getContext())
            .inflate(R.layout.item_tarea, parent, false);
        return new TareaViewHolder(vista);
    }

    // ----- 2. ENLAZAR: se llama CONSTANTEMENTE -----
    @Override
    public void onBindViewHolder(@NonNull TareaViewHolder holder, int position) {
        holder.enlazar(lista.get(position)); // solo asignar datos
    }

    // ----- 3. CONTAR -----
    @Override
    public int getItemCount() {
        return lista.size();
    }

    // =====
    class TareaViewHolder extends RecyclerView.ViewHolder {

        private final TextView tvTitulo;
        private final TextView tvMateria;
```

```

private final TextView tvFecha;
private final View indicadorPrioridad;

TareaViewHolder(@NonNull View itemView) {
    super(itemView);

    // findViewById UNA sola vez por vista creada, no una por elemento mostrado.
    tvTitulo = itemView.findViewById(R.id.tvTitulo);
    tvMateria = itemView.findViewById(R.id.tvMateria);
    tvFecha = itemView.findViewById(R.id.tvFecha);
    indicadorPrioridad = itemView.findViewById(R.id.indicadorPrioridad);

    // Los escuchadores también se registran UNA vez, aquí y no en onBind.
    itemView.setOnClickListener(v -> {
        int pos = getBindingAdapterPosition();
        if (pos != RecyclerView.NO_POSITION) {
            listener.onTareaClick(lista.get(pos), pos);
        }
    });

    itemView.setOnLongClickListener(v -> {
        int pos = getBindingAdapterPosition();
        if (pos != RecyclerView.NO_POSITION) {
            listener.onTareaLongClick(lista.get(pos), pos);
        }
        return true; // true = evento consumido; evita disparar el clic simple
    });
}

void enlazar(Tarea tarea) {
    tvTitulo.setText(tarea.getTitulo());
    tvMateria.setText(tarea.getMateria());
    tvFecha.setText(itemView.getContext()
        .getString(R.string.formato_fecha_entrega, tarea.getFechaEntrega()));

    indicadorPrioridad.setBackgroundColor(
        ContextCompat.getColor(itemView.getContext(),
            tarea.getPrioridad().getColorRes()));
}
}

```

LAS TRES DECISIONES QUE HAY QUE PODER JUSTIFICAR DE ESTE ARCHIVO

- 1. Los escuchadores van en el constructor del ViewHolder, no en onBindViewHolder.** Si se registraran en `onBind`, se crearía un objeto lambda nuevo en cada desplazamiento: decenas de objetos por segundo, presión inútil sobre el recolector de basura.
- 2. Se usa `getBindingAdapterPosition()` y no el parámetro `position`.** El parámetro `position` es válido solo *durante* el enlace. Cuando el usuario pulse, quizá pasaron segundos y se insertaron o eliminaron elementos: `position` estaría obsoleto y se abriría la tarea equivocada. `getBindingAdapterPosition()` consulta la posición **actual**.
- 3. Se comprueba `NO_POSITION`.** Si el ViewHolder fue reciclado o eliminado justo antes del clic, el método devuelve `RecyclerView.NO_POSITION` (que vale `-1`). Sin esa comprobación, `lista.get(-1)` lanza `IndexOutOfBoundsException`. Este fallo aparece justo al borrar elementos: es el error más frecuente en esta práctica.

Paso 5 • `res/layout/fragment_tareas.xml`

Aunque la lista se construya primero dentro de `MainActivity`, conviene crearla ya en este layout: en AP-3.8.2 pasará a ser el contenido del fragmento de Tareas sin tener que rehacerla.

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/rvTareas"
        android:layout_width="0dp"
        android:layout_height="0dp"
        android:clipToPadding="false"
        android:paddingBottom="@dimen/espaciado_lista_inferior"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <com.google.android.material.floatingactionbutton.FloatingActionButton
        android:id="@+id/fabNuevaTarea"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_margin="@dimen/espaciado_medio"
        android:contentDescription="@string/fab_nueva_tarea"
        android:src="@drawable/ic_agregar"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

```

<!-- res/values/dimens.xml -->
<dimen name="espaciado_lista_inferior">88dp</dimen>

```

`clipToPadding="false"` junto con `paddingBottom` hace que el último elemento de la lista pueda desplazarse **por encima** del relleno, de modo que el FAB no tape la última tarjeta. Sin esos dos atributos, la última tarjeta queda permanentemente oculta bajo el botón: es un defecto visible que cuesta dos atributos corregir.

Ícono `res/drawable/ic_agregar.xml`

```

<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24"
    android:tint="?attr/colorOnPrimaryContainer">
    <path
        android:pathData="M11,5 H13 V11 H19 V13 H11 V13 H5 V11 H11 Z"
        android:fillColor="@android:color/white" />
</vector>

```

El truco de `android:fillColor="@android:color/white"` combinado con `android:tint="?attr/..."` es el idioma habitual de los íconos vectoriales: el trazo se dibuja en blanco y el tinte del tema lo recolorea. Así el mismo archivo sirve en modo claro y oscuro.

Paso 6 · MainActivity.java (versión de AP-3.8.1)

```

package com.example.miagendaintjem;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.widget.AdapterView;
import android.widget.AdapterView.OnItemClickListener;
import android.widget.AdapterView.OnItemSelectedListener;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.example.miagendaintjem.adaptador.TareaAdapter;
import com.example.miagendaintjem.modelo.Prioridad;
import com.example.miagendaintjem.modelo.Tarea;
import com.google.android.material.dialog.MaterialAlertDialogBuilder;
import com.google.android.material.floatingactionbutton.FloatingActionButton;
import com.google.android.material.textfield.TextInputEditText;

import java.text.SimpleDateFormat;
import java.util.ArrayList;

```

```

import java.util.Calendar;
import java.util.List;
import java.util.Locale;

public class MainActivity extends AppCompatActivity
    implements TareaAdapter.OnTareaClickListener {

    private static final String ESTADO_TAREAS = "estado_tareas";

    private final List<Tarea> tareas = new ArrayList<>();
    private TareaAdapter adaptador;
    private RecyclerView rvTareas;
    private int siguienteId = 1;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.fragment_tareas); // en AP-3.8.2 pasará al fragmento

        rvTareas = findViewById(R.id.rvTareas);
        FloatingActionButton fab = findViewById(R.id.fabNuevaTarea);

        if (savedInstanceState != null) {
            List<Tarea> guardadas =
                savedInstanceState.getParcelableArrayList(ESTADO_TAREAS);
            if (guardadas != null) {
                tareas.addAll(guardadas);
                siguienteId = tareas.size() + 1;
            }
        } else {
            cargarTareasDePrueba();
        }

        adaptador = new TareaAdapter(tareas, this);
        rvTareas.setLayoutManager(new LinearLayoutManager(this));
        rvTareas.setAdapter(adaptador);
        rvTareas.setHasFixedSize(true);

        fab.setOnClickListener(v -> mostrarDialogoNuevaTarea());
    }

    @Override
    protected void onSaveInstanceState(@NonNull Bundle outState) {
        super.onSaveInstanceState(outState);
        outState.putParcelableArrayList(ESTADO_TAREAS, new ArrayList<>(tareas));
    }

    // ----- Datos de prueba: 8 tareas, sin literales -----
    private void cargarTareasDePrueba() {
        String[] titulos = getResources().getStringArray(R.array.tareas_demo);
        String[] materias = getResources().getStringArray(R.array.materias);
        Prioridad[] prioridades = Prioridad.values();

        SimpleDateFormat formato =
            new SimpleDateFormat("dd/MM/yyyy", new Locale("es", "BO"));
        Calendar calendario = Calendar.getInstance();

        for (int i = 0; i < titulos.length; i++) {
            calendario.add(Calendar.DAY_OF_YEAR, 3);
            tareas.add(new Tarea(
                siguienteId++,
                titulos[i],
                materias[i % materias.length],
                formato.format(calendario.getTime()),
                prioridades[i % prioridades.length]));
        }
    }

    // ----- Requerimiento 5: clic simple -----
    @Override
    public void onTareaClick(Tarea tarea, int posicion) {
        Toast.makeText(this, tarea.getTitulo(), Toast.LENGTH_SHORT).show();
    }

    // ----- Requerimiento 7: pulsación larga + confirmación -----
    @Override
    public void onTareaLongClick(Tarea tarea, int posicion) {
        new MaterialAlertDialogBuilder(this)
            .setTitle(R.string.dialogo_eliminar_titulo)
            .setMessage(getString(R.string.dialogo_eliminar_mensaje, tarea.getTitulo()))
            .setNegativeButton(R.string.accion_cancelar, null)
            .setPositiveButton(R.string.accion_eliminar,
                (dialogo, boton) -> eliminarTarea(posicion))
            .show();
    }

    private void eliminarTarea(int posicion) {

```

```

        if (posicion < 0 || posicion >= tareas.size()) return;    // defensa

        tareas.remove(posicion);
        adaptador.notifyItemRemoved(posicion);
        adaptador.notifyItemRangeChanged(posicion, tareas.size() - posicion);

        Toast.makeText(this, R.string.msg_tarea_eliminada, Toast.LENGTH_SHORT).show();
    }

    // ----- Requerimiento 6: FAB + alta de tarea -----
    private void mostrarDialogoNuevaTarea() {
        View vista = LayoutInflater.from(this)
            .inflate(R.layout.dialogo_nueva_tarea, null);

        TextInputEditText etTitulo = vista.findViewById(R.id.etTitulo);
        Spinner spMateria = vista.findViewById(R.id.spMateria);
        Spinner spPrioridad = vista.findViewById(R.id.spPrioridad);

        llenarSpinner(spMateria, R.array.materias);
        llenarSpinner(spPrioridad, R.array.prioridades);

        new MaterialAlertDialogBuilder(this)
            .setTitle(R.string.dialogo_nueva_titulo)
            .setView(vista)
            .setNegativeButton(R.string.accion_cancelar, null)
            .setPositiveButton(R.string.accion_guardar, (dialogo, boton) -> {
                CharSequence entrada = etTitulo.getText();
                String titulo = entrada == null ? "" : entrada.toString().trim();

                if (titulo.isEmpty()) {
                    Toast.makeText(this, R.string.error_titulo_vacio,
                        Toast.LENGTH_SHORT).show();
                    return;
                }
                agregarTarea(titulo,
                    (String) spMateria.getSelectedItem(),
                    Prioridad.values()[spPrioridad.getSelectedItemPosition()]);
            })
            .show();
    }

    private void llenarSpinner(Spinner spinner, int arrayRes) {
        ArrayAdapter<CharSequence> adaptadorSpinner = ArrayAdapter.createFromResource(
            this, arrayRes, android.R.layout.simple_spinner_item);
        adaptadorSpinner.setDropDownViewResource(
            android.R.layout.simple_spinner_dropdown_item);
        spinner.setAdapter(adaptadorSpinner);
    }

    private void agregarTarea(String titulo, String materia, Prioridad prioridad) {
        SimpleDateFormat formato =
            new SimpleDateFormat("dd/MM/yyyy", new Locale("es", "BO"));
        Calendar calendario = Calendar.getInstance();
        calendario.add(Calendar.DAY_OF_YEAR, 7);

        tareas.add(new Tarea(siguieteId++, titulo, materia,
            formato.format(calendario.getTime()), prioridad));

        int posicion = tareas.size() - 1;
        adaptador.notifyItemInserted(posicion);    // NO notifyDataSetChanged
        rvTareas.scrollToPosition(posicion);        // desplazar al nuevo elemento
    }
}

```

res/layout/dialogo_nueva_tarea.xml

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="@dimen/espaciado_grande">

    <com.google.android.material.textfield.TextInputLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="@string/campo_titulo">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/etTitulo"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textCapSentences" />
        </com.google.android.material.textfield.TextInputLayout>

    <Spinner
        android:id="@+id/spMateria"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="@dimen/espaciado_medio" />

    <Spinner
        android:id="@+id/spPrioridad"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="@dimen/espaciado_medio" />
</LinearLayout>

```

DOS DETALLES DEL CÓDIGO QUE VALE LA PENA SEÑALAR EN EL INFORME

notifyItemRangeChanged **después de** **notifyItemRemoved**. Al eliminar el elemento de la posición 3, los que estaban en 4, 5, 6... pasan a 3, 4, 5... Sus ViewHolder siguen teniendo la posición antigua guardada. La llamada a `notifyItemRangeChanged(posicion, tareas.size() - posicion)` los reenlaza. Sin ella, los clics posteriores pueden apuntar al elemento equivocado.

setHasFixedSize(true). Le comunica al `RecyclerView` que su tamaño no depende del contenido del adaptador, lo que le permite saltarse una medición completa en cada cambio. Es correcto aquí porque la lista ocupa un área fija de la pantalla.

AP-3.8.2 · Navegación: Activities, Intents y Fragments

LO QUE LA GUÍA ANUNCIA QUE VA A PREGUNTAR

La consigna dice textualmente que en la sustentación hay que explicar **la diferencia entre un Intent explícito y uno implícito** y **qué ocurre con el estado de un Fragment cuando se rota el dispositivo**, y añade que es «el contenido que con mayor frecuencia se reproduce sin comprensión». Las dos respuestas están desarrolladas en el capítulo 2 de este documento. Conviene volver a leerlas después de escribir el código, cuando ya se tenga el referente concreto.

Esta actividad transforma una aplicación de una sola pantalla en una aplicación navegable. Hay tres movimientos independientes y conviene no mezclarlos mentalmente:

Movimiento	Mecanismo	Qué demuestra
Ir al detalle y volver con un resultado	Activity + Intent explícito + <code>ActivityResultLauncher</code>	Comunicación entre Activities
Compartir el detalle con otra aplicación	Intent implícito + <code>createChooser</code>	Comunicación entre aplicaciones
Cambiar de sección dentro de la misma pantalla	Fragments + <code>FragmentManager</code>	Composición interna de una Activity

Paso 1 · Prerrequisito del tema: `NoActionBar`

Para poner una `Toolbar` propia hay que quitar antes la barra que el sistema pone por defecto. Si en la guía 3.7 se hizo la actividad complementaria AC-3.7.3, el tema ya es `Theme.Material3.DayNight.NoActionBar` y no hay nada que hacer. Si no se hizo, hay que cambiarlo ahora en `res/values/themes.xml`:

```
<style name="Theme.MiAgendaINTJEM"
    parent="Theme.Material3.DayNight.NoActionBar">
    <!-- ... roles de color ya definidos en la guía 3.7 ... -->
</style>
```

SÍNTOMA SI SE OLVIDA

La aplicación arranca y se cierra con `IllegalStateException: This Activity already has an action bar supplied by the window decor` en cuanto se llama a `setSupportActionBar()`. El mensaje es literal: ya hay una barra, no se puede poner otra.

Paso 2 · Recursos nuevos

Añadir a `res/values/strings.xml`

```
<!-- Navegación -->
<string name="seccion_tareas">Tareas</string>
<string name="seccion_calendario">Calendario</string>
<string name="seccion_perfil">Perfil</string>

<!-- Detalle -->
<string name="titulo_detalle">Detalle de la tarea</string>
<string name="etiqueta_materia">Materia</string>
<string name="etiqueta_fecha">Fecha de entrega</string>
<string name="etiqueta_prioridad">Prioridad</string>
<string name="accion_guardar_cambios">Guardar cambios</string>
<string name="accion_compartir">Compartir</string>
<string name="accion_acerca_de">Acerca de</string>

<!-- Intent implícito -->
<string name="compartir_asunto">Tarea: %1$s</string>
<string name="compartir_cuerpo">Tarea: %1$s\nMateria: %2$s\nEntrega: %3$s\nPrioridad: %4$s</string>
<string name="compartir_titulo_selector">Compartir tarea mediante...</string>

<!-- Mensajes -->
<string name="msg_tarea_actualizada">Tarea actualizada</string>
<string name="msg_sin_cambios">No se realizaron cambios</string>
<string name="desc_icono_seccion">Icono de sección</string>
```

POR QUÉ LAS CADENAS LLEVAN `%1S` < /CODE > Y NO CONCATENACIÓN < /SPAN > < P > < CODE
> `GETSTRING(R.STRING.COMPARTIR_CUERPO,TITULO,MATERIA,FECHA,PRIORIDAD)` < /CODE
> *SUSTITUYE LOS MARCADORES EN ORDEN. LA VENTAJA NO ES ESTÉTICA: EN UN IDIOMA CON OTRO ORDEN SINTÁCTICO, EL TRADUCTOR PUEDE REORDENAR*
< CODE > **S Y `%2S` DENTRO DE LA CADENA SIN TOCAR EL CÓDIGO JAVA. CON "TAREA:" + TITULO + ":" + MATERIA ESO ES IMPOSIBLE. ES EL MISMO ARGUMENTO DE EXTERNALIZACIÓN DE LA GUÍA 3.7, UN NIVEL MÁS ARRIBA.**

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item
        android:id="@+id/nav_tareas"
        android:icon="@drawable/ic_tareas"
        android:title="@string/seccion_tareas" />

    <item
        android:id="@+id/nav_calendario"
        android:icon="@drawable/ic_calendario"
        android:title="@string/seccion_calendario" />

    <item
        android:id="@+id/nav_perfil"
        android:icon="@drawable/ic_perfil"
        android:title="@string/seccion_perfil" />

</menu>
```

LÍMITE DURO: CINCO ELEMENTOS

`BottomNavigationView` admite como máximo **cinco** elementos. Con más, la especificación de Material manda usar un cajón lateral (`NavigationDrawer`). La guía pide tres, que está dentro del límite. Es una pregunta de sustentación probable: «¿cuántas secciones caben aquí y qué se usa si son más?».

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">

    <item
        android:id="@+id/accion_acerca_de"
        android:title="@string/accion_acerca_de"
        app:showAsAction="never" />

</menu>
```

EL ERROR QUE COMPILA PERO NO FUNCIONA

El atributo es `app:showAsAction`, con el espacio de nombres `app`, no `android:showAsAction`. Al usar `androidx.appcompat`, el atributo del sistema es ignorado y el elemento nunca aparece en la barra por más que se ponga `always`. Es un fallo silencioso: no hay error de compilación ni excepción, simplemente no se ve nada.

Valores posibles: `always` (siempre visible como ícono), `ifRoom` (visible si cabe), `never` (siempre en el menú desplegable de los tres puntos).

Los tres íconos de navegación se crean como drawables vectoriales, igual que `ic_agenda` de la guía 3.7. Se pueden generar desde Android Studio con **File ▶ New ▶ Vector Asset** eligiendo los íconos `check_circle`, `event` y `person` de la biblioteca de Material.

Paso 3 · `DetalleTareaActivity` y el Intent explícito

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="@dimen/espaciado_grande">

    <com.google.android.material.textfield.TextInputLayout
        android:id="@+id/titTitulo"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:hint="@string/campo_titulo"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/etTituloDetalle"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textCapSentences" />
    </com.google.android.material.textfield.TextInputLayout>

    <TextView
        android:id="@+id/tvMateriaDetalle"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="@dimen/espaciado_medio"
        android:textAppearance="@attr/textAppearanceBodyLarge"
        android:textColor="@attr/colorOnSurface"
        app:layout_constraintTop_toBottomOf="@id/titTitulo"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <TextView
        android:id="@+id/tvFechaDetalle"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="@dimen/espaciado_pequeno"
        android:textAppearance="@attr/textAppearanceBodyMedium"
        android:textColor="@attr/colorOnSurfaceVariant"
        app:layout_constraintTop_toBottomOf="@id/tvMateriaDetalle"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <TextView
        android:id="@+id/tvPrioridadDetalle"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="@dimen/espaciado_pequeno"
        android:textAppearance="@attr/textAppearanceLabelLarge"
        app:layout_constraintTop_toBottomOf="@id/tvFechaDetalle"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <com.google.android.material.button.MaterialButton
        android:id="@+id/btnCompartir"
        style="@style/EstiloBotonPrincipal"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="@string/accion_compartir"
        app:layout_constraintBottom_toTopOf="@id/btnGuardar"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:layout_marginBottom="@dimen/espaciado_medio" />

    <com.google.android.material.button.MaterialButton
        android:id="@+id/btnGuardar"
        style="@style/EstiloBotonPrincipal"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="@string/accion_guardar_cambios"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>

```

Nótese que se reutiliza `EstiloBotonPrincipal` de la guía 3.7. La consigna de AP-3.8.1 pedía explícitamente reutilizar los recursos de la guía anterior; esto lo materializa y conviene señalarlo en el informe.

`DetalleTareaActivity.java`

```
package com.example.miagendaintjem;
```

```

import android.content.Intent;
import android.os.Bundle;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.IntentCompat;

import com.example.miagendaintjem.modelo.Tarea;
import com.google.android.material.button.MaterialButton;
import com.google.android.material.textfield.TextInputEditText;

public class DetalleTareaActivity extends AppCompatActivity {

    // Clave cualificada con el paquete: evita colisiones con extras de otras apps
    public static final String EXTRA_TAREA =
        "com.example.miagendaintjem.EXTRA_TAREA";
    public static final String EXTRA_POSICION =
        "com.example.miagendaintjem.EXTRA_POSICION";

    private Tarea tarea;
    private int posicion;
    private TextInputEditText etTitulo;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_detalle_tarea);
        setTitle(R.string.titulo_detalle);

        // ---- Recepción del objeto enviado por el Intent explícito ----
        tarea = IntentCompat.getParcelableExtra(
            getIntent(), EXTRA_TAREA, Tarea.class);
        posicion = getIntent().getIntExtra(EXTRA_POSICION, -1);

        if (tarea == null) { // defensa: nunca confiar en que el extra llegó
            finish();
            return;
        }

        etTitulo = findViewById(R.id.etTituloDetalle);
        TextView tvMateria = findViewById(R.id.tvMateriaDetalle);
        TextView tvFecha = findViewById(R.id.tvFechaDetalle);
        TextView tvPrioridad = findViewById(R.id.tvPrioridadDetalle);

        etTitulo.setText(tarea.getTitulo());
        tvMateria.setText(getString(R.string.formato_materia,
            getString(R.string.etiqueta_materia), tarea.getMateria()));
        tvFecha.setText(getString(R.string.formato_fecha,
            getString(R.string.etiqueta_fecha), tarea.getFechaEntrega()));
        tvPrioridad.setText(tarea.getPrioridad().getEtiqueta());
        tvPrioridad.setTextColor(
            getColor(tarea.getPrioridad().getColor()));

        MaterialButton btnGuardar = findViewById(R.id.btnGuardar);
        MaterialButton btnCompartir = findViewById(R.id.btnCompartir);

        btnGuardar.setOnClickListener(v -> devolverResultado());
        btnCompartir.setOnClickListener(v -> compartirTarea());
    }

    // ----- Requerimiento 2: devolver el resultado -----
    private void devolverResultado() {
        CharSequence entrada = etTitulo.getText();
        String tituloEditado = entrada == null ? "" : entrada.toString().trim();

        if (tituloEditado.isEmpty()) {
            tituloEditado = tarea.getTitulo(); // no permitir vaciarlo
        }

        Tarea editada = new Tarea(
            tarea.getId(), tituloEditado, tarea.getMateria(),
            tarea.getFechaEntrega(), tarea.getPrioridad());

        Intent resultado = new Intent();
        resultado.putExtra(EXTRA_TAREA, editada);
        resultado.putExtra(EXTRA_POSICION, posicion);

        setResult(RESULT_OK, resultado);
        finish();
    }

    // ----- Requerimiento 3: Intent IMPLÍCITO -----
    private void compartirTarea() {
        String cuerpo = getString(R.string.compartir_cuerpo,
            tarea.getTitulo());
    }
}

```

```

        tarea.getMateria(),
        tarea.getFechaEntrega(),
        getString(tarea.getPrioridad().getEtiqueta()));

    Intent envio = new Intent(Intent.ACTION_SEND);
    envio.setType("text/plain");
    envio.putExtra(Intent.EXTRA_SUBJECT,
        getString(R.string.compartir_asunto, tarea.getTitulo()));
    envio.putExtra(Intent.EXTRA_TEXT, cuerpo);

    startActivity(Intent.createChooser(
        envio, getString(R.string.compartir_titulo_selector)));
}

@Override
public void onBackPressed() {
    setResult(RESULT_CANCELED); // salir sin guardar
    super.onBackPressed();
}
}

```

Hay que añadir a `strings.xml` los dos formatos auxiliares:

```

<string name="formato_materia">%1$s: %2$s</string>
<string name="formato_fecha">%1$s: %2$s</string>

```

Y declarar la Activity en `AndroidManifest.xml`, dentro de `<application>`:

```

<activity
    android:name=".DetalleTareaActivity"
    android:exported="false"
    android:parentActivityName=".MainActivity" />

```

LAS TRES DECISIONES DE ESTE ARCHIVO QUE HAY QUE PODER JUSTIFICAR

- 1. `exported="false"`.** Esta Activity solo debe poder abrirse desde dentro de la propia aplicación. Marcarla como no exportada impide que otra aplicación la invoque. Desde API 31 el atributo es *obligatorio* en cualquier componente que declare un `intent-filter`; declararlo siempre es buena práctica aunque no lo sea.
- 2. `IntentCompat.getParcelableExtra(intent, clave, Tarea.class)`.** El método clásico `getParcelableExtra(String)` quedó obsoleto en API 33 porque no verifica el tipo: si el extra contenía otra cosa, el error aparecía después, en el casteo. La versión con la clase comprueba el tipo en el momento de leer. `IntentCompat`, de `androidx.core`, hace que esto funcione también en API 24.
- 3. La clave `EXTRA_TAREA` lleva el nombre del paquete.** Los extras viajan en un `Bundle` compartido. Una clave llamada "tarea" podría colisionar con la de otro componente. La convención de Android es cualificarla con el paquete, exactamente como hace el propio sistema con `android.intent.extra.TEXT`.

EL SETTYPE("TEXT/PLAIN") NO ES UN TEXTO DE INTERFAZ

Alguien puede preguntar por qué no está externalizado en `strings.xml`. La respuesta: `"text/plain"` es una constante del protocolo de *Intents* (un tipo MIME), no una cadena visible para el usuario. Externalizarla sería un error: si un traductor la cambiara, el selector de aplicaciones dejaría de encontrar candidatos. La regla de la guía 3.7 aplica a lo que el usuario lee, no a los identificadores técnicos.

Qué hace realmente `createChooser`

Sin `createChooser`, el sistema resuelve el Intent implícito y, si el usuario ya eligió una aplicación por defecto para `ACTION_SEND` con `text/plain`, la abre directamente sin preguntar. Con `createChooser`, el selector aparece **siempre**. Para una demostración en clase esto importa: es lo que hace visible que el Intent es implícito.

Además, `createChooser` es la forma segura de lanzar un Intent implícito que podría no tener destinatario: si ninguna aplicación puede atenderlo, muestra un diálogo vacío en lugar de lanzar `ActivityNotFoundException`.

Paso 4 · `TareasFragment` — migrar la lista a un fragmento

La lógica de AP-3.8.1 vivía en `MainActivity`. Ahora `MainActivity` pasa a ser el **anfitrión** de tres fragmentos y toda la lógica de la lista se muda a `TareasFragment`. Esta es la tabla de la migración:

En la Activity (AP-3.8.1)	En el Fragment (AP-3.8.2)
setContentView(R.layout.fragment_tareas) en onCreate	inflater.inflate(...) en onCreateView, devolviendo la vista
findViewById(id)	vista.findViewById(id) dentro de onCreateView
this como Context	requireContext()
this como LifecycleOwner / anfitrión de diálogos	requireActivity() para el diálogo, this para el listener
startActivity(intent)	igual: el fragmento delega en su Activity
onSaveInstanceState(Bundle)	igual, pero en el fragmento

```

package com.example.miagendaintjem.fragments;

import android.content.Intent;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Toast;

import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.core.content.IntentCompat;
import androidx.fragment.app.Fragment;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.example.miagendaintjem.DetalleTareaActivity;
import com.example.miagendaintjem.R;
import com.example.miagendaintjem.adaptador.TareaAdapter;
import com.example.miagendaintjem.modelo.Tarea;
import com.google.android.material.floatingactionbutton.FloatingActionButton;

import java.util.ArrayList;
import java.util.List;

public class TareasFragment extends Fragment
    implements TareaAdapter.OnTareaClickListener {

    private static final String ESTADO_TAREAS = "estado_tareas";

    private final List<Tarea> tareas = new ArrayList<>();
    private TareaAdapter adaptador;
    private RecyclerView rvTareas;
    private int siguienteId = 1;

    // Posición que se está editando; se guarda entre ida y vuelta
    private int posicionEnEdicion = RecyclerView.NO_POSITION;

    private ActivityResultLauncher<Intent> lanzadorDetalle;

    // ----- 1. El registro va en onCreate, NO más tarde -----
    @Override
    public void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        lanzadorDetalle = registerForActivityResult(
            new ActivityResultContracts.StartActivityForResult(),
            resultado -> {
                if (resultado.getResultCode() != android.app.Activity.RESULT_OK
                    || resultado.getData() == null) {
                    Toast.makeText(requireContext(),
                        R.string.msg_sin_cambios, Toast.LENGTH_SHORT).show();
                    return;
                }
                Tarea editada = IntentCompat.getParcelableExtra(
                    resultado.getData(),
                    DetalleTareaActivity.EXTRA_TAREA,
                    Tarea.class);
                int pos = resultado.getData().getIntExtra(
                    DetalleTareaActivity.EXTRA_POSICION,
                    RecyclerView.NO_POSITION);

                if (editada != null && pos >= 0 && pos < tareas.size()) {
                    tareas.set(pos, editada);
                    adaptador.notifyItemChanged(pos); // solo ese elemento
                    Toast.makeText(requireContext(),
                        R.string.msg_tarea_actualizada,
                        Toast.LENGTH_SHORT).show();
                }
            }
        );
    }

```

```

    });
}

@Nullable @Override
public View onCreateView(@NonNull LayoutInflater inflater,
                        @Nullable ViewGroup container,
                        @Nullable Bundle savedInstanceState) {
    return inflater.inflate(R.layout.fragment_tareas, container, false);
}

@Override
public void onViewCreated(@NonNull View vista,
                        @Nullable Bundle savedInstanceState) {
    super.onViewCreated(vista, savedInstanceState);

    rvTareas = vista.findViewById(R.id.rvTareas);
    FloatingActionButton fab = vista.findViewById(R.id.fabNuevaTarea);

    // ----- 2. Restauración del estado propio -----
    if (savedInstanceState != null) {
        List<Tarea> guardadas =
            savedInstanceState.getParcelableArrayList(ESTADO_TAREAS);
        if (guardadas != null) {
            tareas.clear();
            tareas.addAll(guardadas);
            siguienteId = tareas.size() + 1;
        }
    } else if (tareas.isEmpty()) {
        cargarTareasDePrueba();
    }

    adaptador = new TareaAdapter(tareas, this);
    rvTareas.setLayoutManager(new LinearLayoutManager(requireContext()));
    rvTareas.setAdapter(adaptador);
    rvTareas.setHasFixedSize(true);

    fab.setOnClickListener(v -> mostrarDialogoNuevaTarea());
}

@Override
public void onSaveInstanceState(@NonNull Bundle outState) {
    super.onSaveInstanceState(outState);
    outState.putParcelableArrayList(ESTADO_TAREAS, new ArrayList<>(tareas));
}

// ----- 3. Requerimiento 1: Intent EXPLÍCITO -----
@Override
public void onTareaClick(Tarea tarea, int posicion) {
    posicionEnEdicion = posicion;

    Intent intent = new Intent(requireContext(), DetalleTareaActivity.class);
    intent.putExtra(DetalleTareaActivity.EXTRA_TAREA, tarea);
    intent.putExtra(DetalleTareaActivity.EXTRA_POSICION, posicion);

    lanzadorDetalle.launch(intent);
}

// ----- 4. Evitar la fuga de memoria -----
@Override
public void onDestroyView() {
    super.onDestroyView();
    rvTareas.setAdapter(null); // soltar la referencia a la vista
    rvTareas = null;
}

// ... onTareaLongClick, eliminarTarea, mostrarDialogoNuevaTarea,
// llenarSpinner, agregarTarea y cargarTareasDePrueba se copian
// de MainActivity cambiando `this` por `requireContext()` ...
}

```

CUATRO TRAMPAS DE ESTE ARCHIVO, EN ORDEN DE FRECUENCIA

- 1. Registrar el `ActivityResultLauncher` tarde.** `registerForActivityResult` debe llamarse antes de que el fragmento llegue al estado `STARTED`: en `onCreate` o como campo de instancia. Si se llama dentro de `onViewCreated` o de un `setOnClickListener`, el sistema lanza `LifecycleOwner is attempting to register while current state is RESUMED. LifecycleOwners must call register before they are STARTED`. La razón es que el sistema debe poder volver a conectar la devolución de llamada tras una muerte del proceso, y solo puede hacerlo si el registro ocurre siempre en el mismo punto de la reconstrucción.
- 2. Usar `startActivityForResult`.** Está obsoleto desde AndroidX Activity 1.2. No es una preferencia estilística: la API antigua obligaba a un `switch` sobre `requestCode` en `onActivityResult`, lejos del punto de lanzamiento, y no sobrevivía correctamente a la muerte del proceso. La guía pide explícitamente `ActivityResultLauncher`.
- 3. No anular las vistas en `onDestroyView`.** El fragmento sobrevive a su vista (véase el capítulo 2). Si `rvTareas` queda apuntando a la vista destruida, esa vista, su adaptador y todas sus filas quedan retenidas. Con rotaciones repetidas la fuga se acumula.
- 4. `notifyDataSetChanged()` al volver del detalle.** Cambió *un* elemento; hay que llamar a `notifyItemChanged(pos)`. Reenlazar la lista entera es el error que la guía 3.8 está diseñada para detectar.

Los dos fragmentos restantes

`CalendarioFragment` y `PerfilFragment` no tienen requisitos funcionales. Basta con que existan, tengan su layout y no rompan la navegación. Conviene que muestren algo identificable para la demostración.

```
package com.example.miagendaintjem.fragmentos;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;

import com.example.miagendaintjem.R;

public class CalendarioFragment extends Fragment {

    @Nullable @Override
    public View onCreateView(@NonNull LayoutInflater inflater,
                             @Nullable ViewGroup container,
                             @Nullable Bundle savedInstanceState) {
        return inflater.inflate(R.layout.fragment_calendario, container, false);
    }
}
```

`PerfilFragment` es idéntico cambiando el layout. Los dos layouts son un `ConstraintLayout` con un `TextView` centrado que usa `?attr/textAppearanceHeadlineSmall` y una cadena de `strings.xml`.

ATAJO DE ESCRITURA QUE ADEMÁS ES CORRECTO

El constructor `Fragment(int contentLayoutId)` permite escribir el fragmento en dos líneas: `public CalendarioFragment() { super(R.layout.fragment_calendario); }` y nada más. El sistema infla el layout automáticamente. No es una simplificación didáctica: es la forma recomendada en la documentación actual de AndroidX cuando no hay lógica en `onCreateView`.

Paso 5 • `MainActivity` como anfitrión

`res/layout/activity_main.xml`

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <com.google.android.material.appbar.AppBarLayout
        android:id="@+id/appBar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <com.google.android.material.appbar.MaterialToolbar
            android:id="@+id/toolbar"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize"
            app:title="@string/app_name" />
    </com.google.android.material.appbar.AppBarLayout>

    <!-- Contenedor de fragmentos: lo que cambia al tocar la barra inferior -->
    <FrameLayout
        android:id="@+id/contenedorFragmentos"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        app:layout_behavior="@string/appbar_scrolling_view_behavior" />

    <com.google.android.material.bottomnavigation.BottomNavigationView
        android:id="@+id/navegacionInferior"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom"
        app:menu="@menu/menu_navegacion" />
</androidx.coordinatorlayout.widget.CoordinatorLayout>

```

MainActivity.java (versión de AP-3.8.2)

```

package com.example.miagendaintjem;

import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.Fragment;

import com.example.miagendaintjem.fragmentos.CalendarioFragment;
import com.example.miagendaintjem.fragmentos.PerfilFragment;
import com.example.miagendaintjem.fragmentos.TareasFragment;
import com.google.android.material.appbar.MaterialToolbar;
import com.google.android.material.bottomnavigation.BottomNavigationView;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // ----- Requerimiento 6: Toolbar propia -----
        MaterialToolbar toolbar = findViewById(R.id.toolbar);
        setSupportActionBar(toolbar);

        // ----- Requerimiento 4: BottomNavigationView -----
        BottomNavigationView navegacion = findViewById(R.id.navegacionInferior);
        navegacion.setOnItemSelectedListener(item -> {
            int id = item.getItemId();

            // if/else y no switch: los R.id no son constantes de compilación
            if (id == R.id.nav_tareas) {
                reemplazarFragment(new TareasFragment());
            } else if (id == R.id.nav_calendario) {
                reemplazarFragment(new CalendarioFragment());
            } else if (id == R.id.nav_perfil) {
                reemplazarFragment(new PerfilFragment());
            } else {
                return false;
            }
            return true;    // true = marcar el elemento como seleccionado
        });

        // ----- Requerimiento 5: conservar el estado al rotar -----
        if (savedInstanceState == null) {
            navegacion.setSelectedItemId(R.id.nav_tareas);
        }
    }

    private void reemplazarFragment(Fragment fragmento) {
        getSupportFragmentManager()
            .beginTransaction()
            .replace(R.id.contenedorFragmentos, fragmento)
            .commit();
    }

    @Override
    public boolean onCreateOptionsMenu(@NonNull Menu menu) {
        getMenuInflater().inflate(R.menu.menu_principal, menu);
        return true;
    }

    @Override
    public boolean onOptionsItemSelected(@NonNull MenuItem item) {
        if (item.getItemId() == R.id.accion_acerca_de) {
            Toast.makeText(this, R.string.app_name, Toast.LENGTH_SHORT).show();
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
}

```

LA GUARDA IF (SAVEDINSTANCESTATE == NULL) : LA RESPUESTA A LA PREGUNTA ANUNCIADA

Esta línea es, literalmente, lo que la guía adelantó que va a preguntar. El razonamiento completo:

Al rotar, el sistema destruye y recrea `MainActivity`. Antes de destruirla, `FragmentManager` guarda en el `Bundle` qué fragmentos había, en qué contenedor y con qué estado. Al recrearla, los **restaura solo**, antes de que termine `super.onCreate()`.

Si se llama a `setSelectedItemId` sin la guarda, se crea una instancia *nueva* de `TareasFragment` que reemplaza a la restaurada. El resultado visible: se pierde la posición de desplazamiento, se pierden las tareas añadidas en esta sesión, y si el usuario estaba en Perfil, aparece en Tareas. Con la guarda, la llamada solo ocurre en el primer arranque real.

La forma corta de decirlo en la sustentación: *el `FragmentManager` ya restauró el fragmento; volver a crearlo destruye el trabajo que el sistema acaba de hacer.*

TRES DETALLES QUE EL DOCENTE PUEDE SEÑALAR

`setOnItemSelectedListener`, **no** `setOnNavigationItemSelectedListener`. El segundo quedó obsoleto en Material Components 1.4. Sigue funcionando, pero aparece tachado y resta puntos en una revisión de código.

if/else en vez de `switch`. Desde Gradle 5 los identificadores de `R` en una biblioteca no son `static final`, así que no pueden ser etiquetas de un `switch`. En el módulo de aplicación todavía compila, pero Lint lo marca con la advertencia `NonConstantResourceId`. Escribir if/else desde el principio evita la discusión.

replace y no add. `add` apila fragmentos: el nuevo se dibuja encima del anterior, que sigue vivo y visible por transparencia. El síntoma clásico es «se ven las dos pantallas superpuestas». Para navegación por pestañas se usa `replace`.

Paso 6 · Guion de la demostración en vivo

La guía evalúa «demostración en dispositivo». Este es el recorrido mínimo que cubre los seis requerimientos sin repetirse:

1. **Arranque.** La aplicación abre en Tareas con las ocho tareas de prueba. *Requerimiento 4 parcial.*
2. **Tocar una tarea.** Se abre `DetalleTareaActivity`. *Requerimiento 1: Intent explícito.*
3. **Editar el título y pulsar Guardar cambios.** Vuelve a la lista y esa fila —solo esa— se actualiza. *Requerimiento 2: `ActivityResultLauncher`.*
4. **Volver a entrar y pulsar Compartir.** Aparece el selector del sistema con WhatsApp, Gmail, Mensajes, lo que haya instalado. *Requerimiento 3: Intent implícito.* Conviene tener al menos dos aplicaciones instaladas en el emulador para que el selector no sea trivial.
5. **Tocar Calendario y Perfil.** Cambia el contenido; la Toolbar y la barra inferior permanecen. *Requerimiento 4 completo.*
6. **Volver a Tareas, desplazar hasta abajo y rotar el dispositivo.** La sección sigue siendo Tareas y la posición de desplazamiento se mantiene. *Requerimiento 5.*
7. **Abrir los tres puntos de la Toolbar.** Aparece «Acerca de». *Requerimiento 6.*

EL PASO 6 ES EL QUE HAY QUE ENSAYAR

Rotar delante del docente es arriesgado y por eso mismo es lo que más conviene. Hay que probarlo diez veces antes: rotar en cada una de las tres secciones, rotar con un diálogo abierto, rotar justo después de añadir una tarea. Si algo se rompe, es mejor descubrirlo en casa.

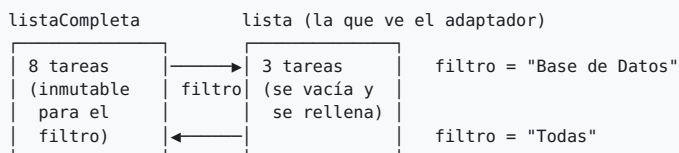
Retos opcionales con puntaje adicional

Dos retos, independientes entre sí. El primero es de dificultad baja y da mucho a cambio de poco; el segundo tiene tres trampas de versión de API que son exactamente lo que un docente busca preguntar.

Reto 1 · Filtrar la lista por materia con un `Spinner`

La idea que hay que entender antes de escribir código

Un filtro no se implementa borrando elementos de la lista. Se implementan **dos** listas:



El adaptador siempre apunta a la **misma** instancia de `lista`. Filtrar significa vaciarla y volver a llenarla, no crear una lista nueva. Si se crea una lista nueva y se le pasa al adaptador, el adaptador antiguo sigue apuntando a la anterior y no pasa nada en pantalla.

Recursos

En `arrays.xml` hace falta un array que incluya la opción «todas». Como `materias` ya existe, se define uno derivado:

```
<string-array name="materias_filtro">
  <item>@string/filtro_todas</item>
  <item>Programación de Dispositivos Móviles</item>
  <item>Base de Datos II</item>
  <item>Ingeniería de Software</item>
  <item>Redes de Computadoras</item>
</string-array>
```

```
<string name="filtro_todas">Todas las materias</string>
<string name="desc_filtro">Filtrar por materia</string>
```

En `fragment_tareas.xml`

El `Spinner` va encima del `RecyclerView`, y el `RecyclerView` pasa a anclarse debajo de él:

```
<Spinner
    android:id="@+id/spFiltroMateria"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_margin="@dimen/espaciado_medio"
    android:contentDescription="@string/desc_filtro"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />
```

En `TareasFragment.java`

```
// Campo nuevo: la lista de origen, que el filtro nunca modifica
private final List<Tarea> listaCompleta = new ArrayList<>();

private void configurarFiltro(View vista) {
    Spinner spFiltro = vista.findViewById(R.id.spFiltroMateria);

    ArrayAdapter<CharSequence> adaptadorFiltro =
        ArrayAdapter.createFromResource(
            requireContext(),
            R.array.materias_filtro,
            android.R.layout.simple_spinner_item);
    adaptadorFiltro.setDropDownViewResource(
        android.R.layout.simple_spinner_dropdown_item);
    spFiltro.setAdapter(adaptadorFiltro);

    spFiltro.setOnItemSelectedListener(new AdapterView.OnItemSelectedListener() {
        @Override
        public void onItemSelected(AdapterView<?> padre, View v,
            int posicion, long id) {
            aplicarFiltro(posicion == 0 ? null
                : (String) padre.getItemAtPosition(posicion));
        }

        @Override
        public void onNothingSelected(AdapterView<?> padre) { /* sin acción */ }
    });
}

private void aplicarFiltro(String materia) {
    tareas.clear(); // misma instancia, se vacía

    if (materia == null) {
        tareas.addAll(listaCompleta); // "Todas"
    } else {
        for (Tarea t : listaCompleta) {
            if (t.getMateria().equals(materia)) {
                tareas.add(t);
            }
        }
    }
    adaptador.notifyDataSetChanged(); // aquí SÍ es correcto
}
```

EL ÚNICO LUGAR DEL PROYECTO DONDE NOTIFYDATASETCHANGED() ES CORRECTO

En todo el resto del código está prohibido. Aquí no: el filtro cambia el conjunto completo de elementos visibles, no una fila. Reenlazar todo es exactamente lo que hay que hacer.

Si el docente pregunta «¿no dijiste que `notifyDataSetChanged` era el error?», la respuesta es: el error es usarlo *para un cambio puntual*, porque borra la información de qué cambió y el `ItemAnimator` no puede animar la transición. Cuando cambia el conjunto entero, no hay información que preservar. La alternativa fina sería `DiffUtil`, que calcula el diferencial entre las dos listas y emite las notificaciones granulares automáticamente.

DOS EFECTOS SECUNDARIOS QUE HAY QUE PREVER

Añadir una tarea con el filtro activo. `agregarTarea` debe insertar en `listaCompleta` y reaplicar el filtro; si solo inserta en `tareas`, la tarea desaparece al cambiar de filtro.

Eliminar con el filtro activo. La posición que llega desde el adaptador es la posición en la *lista filtrada*, no en `listaCompleta`. Hay que eliminar el objeto (`listaCompleta.remove(tarea)`), no el índice. Este error es sutil y borra la tarea equivocada.

Reto 2 · Notificación local para tareas de prioridad alta

Los tres cambios de versión que hay que conocer

API	Versión	Qué cambió	Consecuencia si se ignora
26	Android 8.0	Toda notificación debe pertenecer a un canal	La notificación no se muestra; no hay error visible
31	Android 12	Todo <code>PendingIntent</code> debe declarar <code>FLAG_IMMUTABLE</code> o <code>FLAG_MUTABLE</code>	<code>IllegalArgumentException</code> al construirlo
33	Android 13	Permiso <code>POST_NOTIFICATIONS</code> en tiempo de ejecución	La notificación se descarta silenciosamente

El `minSdk` del proyecto es 24 y el `targetSdk` es 34, así que hay que manejar **los tres**. Este es el motivo real por el que este reto vale puntaje extra.

Permiso en `AndroidManifest.xml`

```
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
```

`notificaciones/NotificadorTareas.java`

```

package com.example.miagendaintjem.notificaciones;

import android.Manifest;
import android.app.Notification;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.app.PendingIntent;
import android.content.Context;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.os.Build;

import androidx.core.app.NotificationCompat;
import androidx.core.app.NotificationManagerCompat;
import androidx.core.content.ContextCompat;

import com.example.miagendaintjem.DetalleTareaActivity;
import com.example.miagendaintjem.MainActivity;
import com.example.miagendaintjem.R;
import com.example.miagendaintjem.modelo.Tarea;

public final class NotificadorTareas {

    public static final String CANAL_PRIORIDAD_ALTA = "canal_prioridad_alta";
    private static int siguienteIdNotificacion = 1000;

    private NotificadorTareas() { } // clase de utilidad: no instanciable

    // ----- 1. Crear el canal (API 26+) -----
    public static void crearCanal(Context contexto) {
        if (Build.VERSION.SDK_INT < Build.VERSION_CODES.O) return;

        NotificationChannel canal = new NotificationChannel(
            CANAL_PRIORIDAD_ALTA,
            contexto.getString(R.string.canal_nombre),
            NotificationManager.IMPORTANCE_HIGH);
        canal.setDescription(contexto.getString(R.string.canal_descripcion));

        NotificationManager gestor =
            contexto.getSystemService(NotificationManager.class);
        gestor.createNotificationChannel(canal); // idempotente
    }

    // ----- 2. Emitir la notificación -----
    public static void notificarTareaAlta(Context contexto, Tarea tarea) {

        Intent intent = new Intent(contexto, MainActivity.class);
        intent.putExtra(DetalleTareaActivity.EXTRA_TAREA, tarea);
        intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK
            | Intent.FLAG_ACTIVITY_CLEAR_TASK);

        PendingIntent pendiente = PendingIntent.getActivity(
            contexto,
            0,
            intent,
            PendingIntent.FLAG_UPDATE_CURRENT
            | PendingIntent.FLAG_IMMUTABLE); // obligatorio desde API 31

        Notification notificacion =
            new NotificationCompat.Builder(contexto, CANAL_PRIORIDAD_ALTA)
                .setSmallIcon(R.drawable.ic_agenda)
                .setContentTitle(
                    contexto.getString(R.string.notif_titulo))
                .setContentText(
                    contexto.getString(R.string.notif_texto,
                        tarea.getTitulo(), tarea.getMateria()))
                .setPriority(NotificationCompat.PRIORITY_HIGH) // pre-26
                .setContentIntent(pendiente)
                .setAutoCancel(true)
                .build();

        // ----- 3. Comprobar el permiso (API 33+) -----
        boolean permitido =
            Build.VERSION.SDK_INT < Build.VERSION_CODES.TIRAMISU
            || ContextCompat.checkSelfPermission(contexto,
                Manifest.permission.POST_NOTIFICATIONS)
                == PackageManager.PERMISSION_GRANTED;

        if (!permitido) return; // sin permiso: no hacer nada, no reventar

        NotificationManagerCompat.from(contexto)
            .notify(siguienteIdNotificacion++, notificacion);
    }
}

```

Solicitar el permiso desde el fragmento

```
private ActivityResultLauncher<String> lanzadorPermiso;

// dentro de onCreate, junto al otro registro
lanzadorPermiso = registerForActivityResult(
    new ActivityResultContracts.RequestPermission(),
    concedido -> { /* si se deniega, la app sigue funcionando igual */ });

private void pedirPermisoNotificaciones() {
    if (Build.VERSION.SDK_INT < Build.VERSION_CODES.TIRAMISU) return;

    if (ContextCompat.checkSelfPermission(requireContext(),
        Manifest.permission.POST_NOTIFICATIONS)
        != PackageManager.PERMISSION_GRANTED) {
        lanzadorPermiso.launch(Manifest.permission.POST_NOTIFICATIONS);
    }
}
```

Y en `agregarTarea`, después de insertar:

```
if (prioridad == Prioridad.ALTA) {
    NotificadorTareas.notificarTareaAlta(requireContext(), nueva);
}
```

El canal se crea una sola vez, al arrancar la aplicación. El lugar canónico es una clase `Application`:

```
public class MiAgendaApp extends android.app.Application {
    @Override
    public void onCreate() {
        super.onCreate();
        NotificadorTareas.crearCanal(this);
    }
}
```

```
<application
    android:name=".MiAgendaApp"
    ... >
```

POR QUÉ FLAG_IMMUTABLE Y NO FLAG_MUTABLE

Un `PendingIntent` es un permiso que la aplicación entrega a otro proceso —aquí, al servicio de notificaciones del sistema— para que ejecute algo *en nombre de ella*. Si es mutable, quien lo recibe puede rellenar campos vacíos del Intent antes de dispararlo. Eso abrió una familia de vulnerabilidades y por eso Android 12 obligó a declararlo explícitamente.

La regla práctica: `FLAG_IMMUTABLE` siempre, salvo que se use una API que necesite rellenar el Intent (respuestas directas en la notificación, burbujas). Aquí el Intent está completo, así que inmutable.

COMPROBACIÓN OBLIGATORIA ANTES DE LA DEMOSTRACIÓN

Si el emulador es API 33 o superior y nunca se aceptó el diálogo de permiso, la notificación no aparece **y no hay ningún mensaje de error**. Antes de demostrar: desinstalar la aplicación, reinstalarla, aceptar el permiso cuando lo pida. Si se quiere provocar el diálogo, `pedirPermisoNotificaciones()` se puede llamar desde `onViewCreated` la primera vez.

Los tres criterios de rechazo, explicados

La guía es categórica: «se devuelve la práctica sin calificar» en tres casos. No son sugerencias de estilo; conviene entender por qué cada uno invalida el trabajo, porque la pregunta «¿por qué está prohibido esto?» es de respuesta obligada en la sustentación.

Criterio 1 · Inflar una vista dentro de `onBindViewHolder`

```
// INCORRECTO – anula todo el mecanismo
@Override
public void onBindViewHolder(@NonNull TareaViewHolder holder, int position) {
    View v = LayoutInflater.from(holder.itemView.getContext())
        .inflate(R.layout.item_tarea, null);
    ...
}
```

`onCreateViewHolder` se llama unas trece veces en toda la vida de la lista; `onBindViewHolder` se llama en cada desplazamiento, decenas de veces por segundo. Inflar dentro de `onBind` significa hacer análisis de XML, reflexión y creación de objetos en cada fotograma. El resultado medible es pérdida de fotogramas al desplazar y presión sobre el recolector de basura.

Dicho de forma corta y defendible: **el reciclado consiste precisamente en no crear vistas al desplazar**. Un adaptador que infla en `onBind` es un `ListView` sin `ViewHolder` disfrazado de `RecyclerView`. Por eso la guía lo rechaza sin calificar: no es un error de implementación, es la ausencia del concepto que la actividad evalúa.

Criterio 2 · Valores literales en `item_tarea.xml`

Es la continuidad directa de la guía 3.7. Un literal en el layout de la fila rompe tres cosas a la vez: la traducción (`values-en` no lo alcanza), el modo oscuro (un color fijo no responde a `values-night`) y la accesibilidad (`sp` fijo en píxeles ignora el ajuste de fuente del sistema).

Lint lo detecta con `HardcodedText`, `PxUsage` y `SpUsage`. **Analyze ► Inspect Code** sobre `res/layout` antes de entregar es literalmente gratis.

Criterio 3 · `LinearLayout` dentro de `ScrollView`

```
<!-- INCORRECTO – no es una lista, es un formulario largo -->
<ScrollView>
    <LinearLayout android:orientation="vertical">
        <!-- ocho tarjetas escritas a mano o añadidas con addView -->
    </LinearLayout>
</ScrollView>
```

Con ocho tareas funciona y se ve igual. Ese es justamente el problema: el estudiante concluye que son equivalentes. No lo son, y la diferencia es de orden de crecimiento:

	<code>ScrollView</code> + <code>LinearLayout</code>	<code>RecyclerView</code>
Vistas creadas para n elementos	n	~13, constante
Memoria	$O(n)$	$O(1)$
Tiempo del primer dibujado	$O(n)$	$O(1)$
Con 1000 elementos	inusable o <code>OutOfMemoryError</code>	idéntico a con 8

La guía usa ocho tareas porque es suficiente para demostrar el patrón, no porque ocho sea la escala esperada. La pregunta de sustentación que persigue este criterio es: «**con ocho elementos las dos soluciones se ven igual; ¿por qué una está rechazada?**» La respuesta es la columna derecha de esa tabla.

Errores frecuentes y diagnóstico

Síntoma	Causa más probable	Corrección
La lista aparece vacía, sin error	Falta <code>setLayoutManager</code>	Asignar <code>LinearLayoutManager</code> antes o después del adaptador, pero asignarlo
<code>IndexOutOfBoundsException: Inconsistency detected</code>	Se modificó la lista sin notificar, o se notificó con una posición vieja	Toda mutación de la lista va seguida del <code>notify*</code> correspondiente, en el mismo método
Al eliminar, se borra la fila equivocada	Se usó <code>position</code> de <code>onBindViewHolder</code> dentro del listener	Usar <code>getBindingAdapterPosition()</code> en el momento del clic y comprobar <code>NO_POSITION</code>
Los datos se mezclan entre filas al desplazar	<code>onBindViewHolder</code> no asigna <i>todos</i> los campos	<code>onBind</code> debe escribir cada vista de la fila en cada llamada, incluso para restaurar valores por defecto
La app se cierra al rotar	<code>findViewById</code> devuelve <code>null</code> porque falta un <code>id</code> en <code>layout-land</code>	Los calificadores producen variantes del mismo recurso: los <code>id</code> deben coincidir (guía 3.7)
<code>IllegalStateException: This Activity already has an action bar</code>	El tema no es <code>NoActionBar</code>	Cambiar el <code>parent</code> del tema
El elemento del menú no aparece	<code>android:showAsAction</code> en vez de <code>app:showAsAction</code>	Usar el espacio de nombres <code>app</code>
<code>LifecycleOwners must call register before they are STARTED</code>	<code>registerForActivityResult</code> dentro de un listener o de <code>onViewCreated</code>	Moverlo a <code>onCreate</code> o a la inicialización del campo
Al rotar se pierde la sección seleccionada	Falta la guarda <code>if (savedInstanceState == null)</code>	Añadirla alrededor de <code>setSelectedItemId</code>
Dos pantallas superpuestas	<code>add</code> en vez de <code>replace</code>	Usar <code>replace</code> en la transacción
<code>ClassCastException</code> al leer el extra	El orden de <code>writeToParcel</code> no coincide con el del constructor <code>Parcel</code>	Revisar campo por campo, en el mismo orden
La notificación no aparece en API 33+	Permiso <code>POST_NOTIFICATIONS</code> no concedido	Solicitarlo con <code>RequestPermission</code> y aceptar
<code>IllegalArgumentException</code> al crear el <code>PendingIntent</code>	Falta <code>FLAG_IMMUTABLE</code> en API 31+	Añadirlo con <code> </code> al resto de banderas
Memoria que crece con cada rotación	Vistas retenidas tras <code>onDestroyView</code>	Anular las referencias a vistas en <code>onDestroyView</code>

Guion de exposición

Duración objetivo: **10 minutos**. Los dos integrantes deben poder dar cualquiera de las dos mitades: la guía asigna un 15 % a la «sustentación individual dentro de la pareja» con preguntas cruzadas del docente, lo que significa que se pregunta a uno y la nota es de los dos.

Minuto 0:00 - 1:00 · Apertura: el problema

«El punto 3.7 resolvió cómo se declara una interfaz. El 3.8 resuelve dos problemas nuevos: cómo mostrar una cantidad arbitraria de elementos sin quedarse sin memoria, y cómo pasar de una pantalla a otra sin perder el estado. Los dos se resuelven con el mismo principio: no crear objetos que se pueden reutilizar, y no asumir que un objeto sobrevive a un cambio de configuración.»

Minuto 1:00 - 2:30 · El reciclado, con el número

Aquí va la cifra. Es lo que distingue una exposición entendida de una recitada.

«Una lista de mil tareas con diez visibles. Sin reciclado: mil objetos `View`, cada uno con su árbol interno, y cuatro mil llamadas a `findViewById`. Con `RecyclerView`: unos trece `ViewHolder` —los diez visibles más los de los bordes—, cincuenta y dos `findViewById`, y esos trece objetos se reutilizan durante todo el desplazamiento. La memoria deja de depender del número de elementos.»

Dibujar en la pizarra, si hay: la ventana visible, el elemento que sale por arriba, la flecha al `RecycledViewPool`, el elemento que entra por abajo tomando ese mismo `ViewHolder`.

Minuto 2:30 - 4:00 · Adapter-ViewHolder en tres métodos

«`onCreateViewHolder` infla la fila y busca sus vistas: se llama unas trece veces. `onBindViewHolder` escribe los datos en una fila existente: se llama en cada desplazamiento, así que ahí dentro no puede haber inflado ni `findViewById`. `getItemCount` devuelve el tamaño de la lista. El `ViewHolder` existe precisamente para que `findViewById` ocurra una vez por fila y no una vez por enlace.»

Mostrar `TareaAdapter.java` en pantalla mientras se dice esto.

Minuto 4:00 - 5:30 · Demostración: la lista

Ejecutar la aplicación. Desplazar. Añadir una tarea con el FAB. Mantener pulsada una tarea y confirmar la eliminación.

«Al añadir uso `notifyItemInserted`, no `notifyDataSetChanged`. La diferencia es que el primero le dice al `RecyclerView` qué cambió, y eso le permite animar la inserción y reenlazar solo lo necesario.»

Minuto 5:30 - 7:00 · Demostración: la navegación

Tocar una tarea → detalle → editar → guardar → volver.

«Esto es un Intent explícito: nombro la clase de destino, `DetalleTareaActivity.class`. El objeto `Tarea` viaja como extra porque implementa `Parcelable`. La vuelta usa `ActivityResultLauncher`, que reemplaza a `startActivityForResult`.»

Entrar otra vez y tocar Compartir.

«Esto es un Intent implícito: no nombro ninguna clase, declaro una acción, `ACTION_SEND`, y un tipo MIME, `text/plain`. El sistema busca qué aplicaciones declararon un `intent-filter` compatible y muestra el selector. Yo no sé, ni necesito saber, qué aplicaciones hay instaladas.»

Minuto 7:00 - 8:00 · Fragments y rotación

Cambiar entre las tres secciones. Volver a Tareas, desplazar, **rotar**.

«Las tres secciones son Fragments dentro de un único contenedor; la Toolbar y la barra inferior no se recrean. Al rotar, el `FragmentManager` guarda qué fragmento estaba y lo restaura solo. Por eso el `setSelectedItemId` inicial está dentro de un `if (savedInstanceState == null)`: sin esa guarda estaría creando un fragmento nuevo encima del que el sistema acaba de restaurar, y perdería el desplazamiento y las tareas añadidas.»

Minuto 8:00 - 9:00 · Los cuatro componentes

«Activity es el punto de entrada con interfaz. Service ejecuta trabajo sin interfaz. BroadcastReceiver responde a eventos del sistema. ContentProvider expone datos a otras aplicaciones. Los cuatro se declaran en el Manifest; los tres primeros con etiqueta propia, y desde API 26 los receptores implícitos deben registrarse en código, no en el Manifest. En este proyecto usamos Activity, y el reto opcional de notificaciones se apoya en el `NotificationManager` del sistema.»

Minuto 9:00 - 10:00 · Cierre y trazabilidad

«Los roles están invertidos respecto a la guía 3.7, como pide la consigna, y el historial de commits del repositorio lo refleja. El proyecto reutiliza los colores, dimensiones, cadenas y el estilo `EstiloBotonPrincipal` definidos en la guía anterior: no hay ni un valor literal en `item_tarea.xml`.»

LAS DOS FRASES QUE HAY QUE TENER GRABADAS LITERALMENTE

Explícito vs. implícito: «El explícito nombra la clase de destino y se usa dentro de la aplicación; el implícito declara una acción y un tipo, y el sistema decide quién la atiende. El explícito es obligatorio para componentes propios por seguridad desde Android 5.»

Fragment al rotar: «El `FragmentManager` guarda y restaura el fragmento automáticamente; la instancia es nueva pero el estado de las vistas con `id` se recupera. Lo que no se recupera solo son mis datos, por eso los guardo en `onSaveInstanceState`. Y por eso la creación inicial va dentro de `if (savedInstanceState == null)`.»

Banco de preguntas de defensa

Taparse la columna derecha y responder en voz alta. Si la respuesta no sale en diez segundos, no está aprendida.

Sobre RecyclerView y el reciclado

Pregunta	Respuesta
¿Qué recicla exactamente el <code>RecyclerView</code> ?	Objetos <code>ViewHolder</code> , que contienen la vista de la fila ya inflada y las referencias a sus vistas hijas. No recicla datos.
¿Cuántos <code>ViewHolder</code> existen con 1000 elementos y 10 visibles?	Unos 13: los visibles más los del borde y el caché. La cifra no depende de los 1000.
¿Cuántas veces se llama a <code>onCreateViewHolder</code> ?	Unas 13, al principio. Después, prácticamente ninguna.
¿Cuántas veces a <code>onBindViewHolder</code> ?	Una por cada fila que entra en pantalla, durante todo el desplazamiento.
¿Por qué <code>findViewById</code> va en el <code>ViewHolder</code> y no en <code>onBind</code> ?	Porque <code>findViewById</code> recorre el árbol de vistas. En el <code>ViewHolder</code> se ejecuta una vez por fila creada; en <code>onBind</code> se ejecutaría en cada desplazamiento.
¿Qué hace el <code>LayoutManager</code> ?	Decide la posición geométrica de cada fila y cuándo una vista deja de ser visible, es decir, cuándo se puede reciclar.
¿Qué pasa si no se asigna <code>LayoutManager</code> ?	La lista aparece vacía, sin error ni excepción.
¿Qué es el <code>RecycledViewPool</code> ?	El almacén de <code>ViewHolder</code> totalmente desvinculados, listos para reutilizarse. Guarda 5 por tipo de vista por omisión.
¿Y el caché de vistas?	2 posiciones por omisión, que conservan sus datos: si una fila vuelve inmediatamente, no se reenlaza.
¿Por qué <code>notifyItemInserted</code> y no <code>notifyDataSetChanged</code> ?	El primero indica qué cambió: permite animar y reenlazar solo lo necesario. El segundo invalida todo y pierde esa información.
¿Cuándo sí es correcto <code>notifyDataSetChanged</code> ?	Cuando cambia el conjunto completo, por ejemplo al aplicar un filtro.
¿Qué es <code>DiffUtil</code> ?	Un utilitario que calcula el diferencial entre la lista antigua y la nueva y emite las notificaciones granulares automáticamente.
¿Por qué <code>getBindingAdapterPosition()</code> y no el <code>position</code> del parámetro?	Porque el <code>position</code> que llegó a <code>onBind</code> puede haber quedado obsoleto cuando el usuario toca. El método consulta la posición actual.
¿Qué devuelve si la fila ya no existe?	<code>RecyclerView.NO_POSITION</code> , que vale <code>-1</code> . Hay que comprobarlo antes de indexar la lista.
¿Qué hace <code>setHasFixedSize(true)</code> ?	Declara que el tamaño del <code>RecyclerView</code> no depende del contenido, lo que permite saltarse una medición completa en cada cambio.
Diferencia con <code>ListView</code>	<code>ListView</code> no obligaba al patrón <code>ViewHolder</code> ni separaba la disposición; <code>RecyclerView</code> lo hace obligatorio y delega la geometría en el <code>LayoutManager</code> .

Sobre Activities, Intents y ciclo de vida

Pregunta	Respuesta
Intent explícito	Nombra la clase de destino. Se usa dentro de la propia aplicación.
Intent implícito	Declara acción, y opcionalmente tipo y datos. El sistema resuelve quién lo atiende.
¿Por qué es obligatorio el explícito para componentes propios?	Desde Android 5, un <code>bindService</code> implícito lanza excepción: un Intent implícito podría ser interceptado por otra aplicación.
¿Qué hace <code>createChooser</code> ?	Fuerza la aparición del selector aunque haya una aplicación por omisión, y evita <code>ActivityNotFoundException</code> si no hay ninguna.
¿Para qué sirve <code><queries></code> en el Manifest?	Desde API 30 una aplicación no puede consultar libremente qué otras hay instaladas; <code><queries></code> declara qué Intents se pretende resolver.
¿Cómo viaja un objeto en un Intent?	Serializado. Con <code>Parcelable</code> en Android; <code>Serializable</code> funciona pero usa reflexión y es mucho más lento.
Trampa clásica de <code>Parcelable</code>	El orden de <code>writeToParcel</code> debe coincidir exactamente con el del constructor que lee del <code>Parcel</code> . Si no, <code>ClassCastException</code> o datos cruzados.
¿Por qué <code>ActivityResultLauncher</code> y no <code>startActivityForResult</code> ?	El segundo está obsoleto: separaba el lanzamiento del manejo del resultado y no sobrevivía bien a la muerte del proceso.
¿Dónde debe registrarse el launcher?	En <code>onCreate</code> o como campo, siempre antes del estado <code>STARTED</code> . Si no, <code>IllegalStateException</code> .
Secuencia exacta al rotar una Activity	<code>onPause</code> → <code>onStop</code> → <code>onSaveInstanceState</code> → <code>onDestroy</code> → <code>onCreate</code> → <code>onStart</code> → <code>onRestoreInstanceState</code> → <code>onResume</code> .
¿Qué guarda <code>onSaveInstanceState</code> solo?	El estado de las vistas que tienen <code>android:id</code> . Los datos del programa hay que ponerlos a mano.
¿Cuándo NO se llama a <code>onSaveInstanceState</code> ?	Cuando el usuario cierra la aplicación deliberadamente (botón atrás, deslizar de recientes): ahí el estado no debe sobrevivir.
¿Qué es <code>exported</code> ?	Declara si otro proceso puede invocar el componente. Obligatorio desde API 31 en componentes con <code>intent-filter</code> .

Sobre Fragments

Pregunta	Respuesta
Diferencia esencial con una Activity	El Fragment no es un componente de aplicación: no se declara en el Manifest, no puede existir solo y vive dentro de una Activity.
¿Por qué tiene dos ciclos de vida?	Porque su vista puede destruirse y recrearse sin que el fragmento muera, por ejemplo al entrar en la pila de retroceso.
¿Dónde se busca <code>findViewById</code> ?	En <code>onViewCreated</code> , sobre la vista devuelta por <code>onCreateView</code> .
¿Por qué anular las vistas en <code>onDestroyView</code> ?	Porque el fragmento sobrevive a su vista; una referencia retenida mantiene viva toda la jerarquía destruida.
¿Qué le pasa al Fragment al rotar?	El <code>FragmentManager</code> guarda cuál era y lo restaura. La instancia es nueva; el estado de las vistas con <code>id</code> se recupera; los datos propios solo si se guardaron.
¿Por qué <code>if (savedInstanceState == null)</code> ?	Para no crear un fragmento nuevo encima del que el sistema ya restauró.
<code>replace</code> frente a <code>add</code>	<code>add</code> apila y deja el anterior visible debajo; <code>replace</code> quita el anterior. Para pestañas se usa <code>replace</code> .
¿Qué pone <code>addToBackStack</code> en juego?	Que el botón atrás deshaga la transacción. En navegación por pestañas normalmente no se usa.
¿Cuántos elementos admite <code>BottomNavigationView</code> ?	Máximo 5. Con más, corresponde un cajón lateral.
¿Por qué <code>if/else</code> y no <code>switch</code> sobre <code>R.id</code> ?	Porque los identificadores de recursos no son constantes de compilación en bibliotecas; Lint lo advierte con <code>NonConstantResourceId</code> .

Sobre los cuatro componentes de aplicación

Pregunta	Respuesta
Nómbrales	Activity, Service, BroadcastReceiver, ContentProvider.
¿Qué tienen en común?	Los cuatro son puntos de entrada que el sistema puede instanciar, y los cuatro se declaran en el <code>AndroidManifest</code> .
Activity	Pantalla con interfaz. Etiqueta <code><activity></code> .
Service	Trabajo prolongado sin interfaz. Etiqueta <code><service></code> . Desde API 26 el trabajo en segundo plano está restringido; se usa <code>WorkManager</code> o servicios en primer plano.
BroadcastReceiver	Responde a anuncios del sistema o de la aplicación. Etiqueta <code><receiver></code> . Desde API 26 la mayoría de los implícitos deben registrarse en código.
ContentProvider	Expone datos a otras aplicaciones mediante URIs. Etiqueta <code><provider></code> .
¿Es el Fragment uno de ellos?	No. Es una unidad de interfaz reutilizable dentro de una Activity. Es una pregunta habitual de descarte.

Preguntas con trampa

Pregunta	Respuesta correcta
«Con ocho tareas, <code>ScrollView</code> + <code>LinearLayout</code> se ve igual. ¿Por qué está rechazado?»	Porque el costo crece con n : ocho vistas frente a ~13 constantes. La equivalencia visual con ocho elementos no dice nada del comportamiento con mil.
«¿No decías que <code>notifyDataSetChanged</code> era el error?»	Es el error para un cambio puntual. Para un cambio del conjunto completo, como un filtro, es lo correcto.
«¿Por qué <code>"text/plain"</code> no está en <code>strings.xml</code> ?»	Porque es una constante del protocolo de Intents, no un texto visible. Traducirla rompería la resolución del Intent.
«¿El Fragment se destruye al rotar?»	Se destruye la instancia y el <code>FragmentManager</code> crea otra a partir del estado guardado. Decir «no se destruye» es incorrecto; decir «se pierde» también.
«¿ <code>onSaveInstanceState</code> guarda mi <code>List<Tarea></code> ?»	No por sí solo. Guarda el estado de las vistas con <code>id</code> . La lista hay que escribirla con <code>putParcelableArrayList</code> .
«¿Para qué sirve el <code>ViewHolder</code> si <code>RecyclerView</code> ya recicla?»	El reciclado reutiliza el objeto vista; el <code>ViewHolder</code> evita repetir <code>findViewById</code> sobre esa vista reutilizada. Son dos ahorros distintos.
«¿Qué pasa si el <code>Intent</code> implícito no encuentra a nadie?»	Con <code>startActivity</code> directo, <code>ActivityNotFoundException</code> . Con <code>createChooser</code> , un diálogo vacío.
«Muéstrame dónde infla tu adaptador.»	En <code>onCreateViewHolder</code> , con <code>LayoutInflater.from(parent.getContext()).inflate(R.layout.item_tarea, parent, false)</code> . El <code>false</code> es clave: la fila la adjunta el <code>RecyclerView</code> , no el inflador.

Tarjetas de memorización

Números que hay que saber de memoria

Dato	Valor
<code>ViewHolder</code> vivos con 1000 elementos y 10 visibles	~13
Llamadas a <code>findViewById</code> en ese caso	52 (13 × 4) frente a 4000 sin el patrón
Tamaño del <code>RecycledViewPool</code>	5 por tipo de vista
Tamaño del caché de vistas	2 posiciones
Máximo de elementos en <code>BottomNavigationView</code>	5
Componentes de aplicación	4 (Activity, Service, BroadcastReceiver, ContentProvider)
Métodos obligatorios del Adapter	3 (<code>onCreateViewHolder</code> , <code>onBindViewHolder</code> , <code>getItemCount</code>)
minSdk / targetSdk de esta guía	24 / 34
API en que el canal de notificación se vuelve obligatorio	26
API en que <code>FLAG_IMMUTABLE</code> se vuelve obligatorio	31
API en que <code>POST_NOTIFICATIONS</code> pasa a tiempo de ejecución	33
API en que <code><queries></code> se vuelve necesario	30
Valor de <code>RecyclerView.NO_POSITION</code>	-1
Tareas de prueba que pide la guía	al menos 8
Ponderación de la sustentación individual	15 %

Definiciones en una línea

- RecyclerView** : contenedor que mantiene en memoria solo las vistas visibles y reutiliza los `ViewHolder` que salen de pantalla.
- Adapter** : traduce una posición de la lista de datos a una fila enlazada.
- ViewHolder** : caja que guarda una fila ya inflada junto con las referencias a sus vistas hijas.
- LayoutManager** : decide la geometría de las filas y cuándo una deja de ser visible.
- ItemAnimator** : anima inserciones, eliminaciones y cambios; necesita notificaciones granulares para funcionar.
- Intent explícito** : nombra la clase de destino.
- Intent implícito** : declara acción y tipo; el sistema resuelve el destino.
- Parcelable** : mecanismo de serialización de Android, sin reflexión, para pasar objetos entre componentes.
- ActivityResultLauncher** : API actual para lanzar una Activity y recibir su resultado; se registra antes de `STARTED` .
- Fragment** : unidad de interfaz reutilizable con ciclo de vida propio, alojada en una Activity.
- FragmentManager** : quien crea, reemplaza, guarda y restaura fragmentos.
- Canal de notificación** : categoría obligatoria desde API 26 que el usuario puede silenciar por separado.

Cadena causal que ordena todo el punto 3.8

Una lista puede tener una cantidad arbitraria de elementos

▼

Crear una vista por elemento hace que la memoria crezca con n

▼

Solo son visibles ~10 a la vez → basta con ~13 vistas

▼

RecyclerView reutiliza esas 13 → RecyclerViewPool y caché

▼

Reutilizar una vista exige separar "crear" de "enlazar"

├─ onCreateViewHolder: infla + findViewById (13 veces)

└─ onBindViewHolder: solo escribe datos (miles de veces)

▼

El ViewHolder es lo que hace posible esa separación

▼

Reutilizar exige avisar QUÉ cambió → notifyItem*, no notifyDataSetChanged

▼

Una fila tocada abre otra pantalla → Intent

├─ destino propio y conocido → explícito (obligatorio por seguridad)

└─ destino ajeno y desconocido → implícito (acción + tipo MIME)

▼

El objeto debe viajar entre procesos → Parcelable

▼

Volver con un resultado → ActivityResultLauncher (registrado antes de STARTED)

▼

Varias secciones en una sola Activity → Fragments

▼

Rotar destruye la Activity → el FragmentManager restaura los fragmentos

▼

Por eso la creación inicial va en if (savedInstanceState == null)

▼

Y por eso mis datos propios van en onSaveInstanceState

Lista de verificación de entrega

| Código · AP-3.8.1

- ☐ `Tarea` con los cinco atributos exigidos: `id`, `titulo`, `materia`, `fechaEntrega`, `prioridad`.
- ☐ `item_tarea.xml` usa `MaterialCardView` y muestra título, materia, fecha e indicador de color por prioridad.
- ☐ **Ni un solo literal en `item_tarea.xml`**: verificado con Lint.
- ☐ `TareaAdapter` extiende `RecyclerView.Adapter` con `TareaViewHolder` como clase interna.
- ☐ El inflado ocurre **solo** en `onCreateViewHolder`.
- ☐ Al menos ocho tareas de prueba, mostradas con `LinearLayoutManager`.
- ☐ Clic simple mediante interfaz de escucha propia, con `Toast` del título.
- ☐ `FloatingActionButton` que añade una tarea y notifica con `notifyItemInserted`.
- ☐ Pulsación larga + `AlertDialog` de confirmación + `notifyItemRemoved`.
- ☐ No hay ningún `ScrollView` conteniendo la lista.

| Código · AP-3.8.2

- ☐ `DetalleTareaActivity` declarada en el Manifest con `exported="false"`.
- ☐ Intent explícito con el objeto `Tarea` como extra (`Parcelable`).
- ☐ Retorno con `ActivityResultLauncher` registrado en `onCreate` y `notifyItemChanged` al volver.
- ☐ Intent implícito `ACTION_SEND` con `createChooser`, probado con al menos dos aplicaciones instaladas.
- ☐ `BottomNavigationView` con Tareas, Calendario y Perfil resueltas por Fragments.
- ☐ Reemplazo con `FragmentManager` / `FragmentTransaction` usando `replace`.
- ☐ Guarda `if (savedInstanceState == null)` alrededor de la selección inicial.
- ☐ Estado conservado al rotar: sección, desplazamiento y tareas añadidas.
- ☐ `MaterialToolbar` con `setSupportActionBar` y menú en `res/menu`.
- ☐ Tema `NoActionBar`.
- ☐ Vistas anuladas en `onDestroyView` de los fragmentos.

| Repositorio

- ☐ Historial que incluye los commits de la guía 3.7 y los de la 3.8.
- ☐ Commits de **ambos** integrantes en ambas guías.
- ☐ Los roles están **invertidos** respecto a la guía 3.7 y el historial lo refleja.
- ☐ Nombres de variables y comentarios propios, no copiados de otra pareja ni de este documento.

| Informe

- ☐ Carátula con nombres completos y registro universitario de los dos.
- ☐ Tabla de roles por actividad práctica, indicando la inversión respecto a la 3.7.
- ☐ Cuadro comparativo, diagramas de ciclo de vida y síntesis de la defensa técnica del reciclado.
- ☐ Fragmentos de código relevantes y capturas de ejecución.
- ☐ Dificultades encontradas y cómo se resolvieron.
- ☐ Conclusiones individuales: un párrafo propio y firmado por cada integrante.
- ☐ Anexo con el enlace al repositorio y el historial de commits visible, **incluyendo la guía 3.7**.

| Ensayo de la sustentación

- ☐ Los dos pueden explicar Intent explícito frente a implícito sin leer.
- ☐ Los dos pueden explicar qué le pasa al Fragment al rotar, incluida la guarda `savedInstanceState == null`.
- ☐ Los dos pueden señalar en su propio código dónde se infla y por qué está ahí.
- ☐ Los dos pueden decir la cifra del reciclado y de dónde sale.
- ☐ La rotación se ha probado en las tres secciones y con un diálogo abierto.